FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning

Tri Dao[1,2]

[1]Department of Computer Science, Princeton University
[2]Department of Computer Science, Stanford University
trid@cs.stanford.edu

July 18, 2023

Abstract

Scaling Transformers to longer sequence lengths has been a major problem in the last several years, promising to improve performance in language modeling and high-resolution image understanding, as well as to unlock new applications in code, audio, and video generation. The attention layer is the main bottleneck in scaling to longer sequences, as its runtime and memory increase quadratically in the sequence length. FlashAttention [5] exploits the asymmetric GPU memory hierarchy to bring significant memory saving (linear instead of quadratic) and runtime speedup (2-4× compared to optimized baselines), with no approximation. However, FlashAttention is still not nearly as fast as optimized matrix-multiply (GEMM) operations, reaching only 25-40% of the theoretical maximum FLOPs/s. We observe that the inefficiency is due to suboptimal work partitioning between different thread blocks and warps on the GPU, causing either low-occupancy or unnecessary shared memory reads/writes. We propose FlashAttention-2, with better work partitioning to address these issues. In particular, we (1) tweak the algorithm to reduce the number of non-matmul FLOPs (2) parallelize the attention computation, even for a single head, across different thread blocks to increase occupancy, and (3) within each thread block, distribute the work between warps to reduce communication through shared memory. These yield around 2× speedup compared to FlashAttention, reaching 50-73% of the theoretical maximum FLOPs/s on A100 and getting close to the efficiency of GEMM operations. We empirically validate that when used end-to-end to train GPT-style models, FlashAttention-2 reaches training speed of up to 225 TFLOPs/s per A100 GPU (72% model FLOPs utilization).[1]

1 Introduction

Scaling up the context length of Transformers [18] is a challenge, since the attention layer at their heart has runtime and memory requirements quadratic in the input sequence length. Ideally, we would like to go beyond the standard 2k sequence length limit to train models to understand books, high resolution images, and long-form videos. Just within the last year, there have been several language models with much longer context than before: GPT-4 [12] with context length 32k, MosaicML's MPT with context length 65k, and Anthropic's Claude with context length 100k. Emerging use cases such as long document querying and story writing have demonstrated a need for models with such long context.

To reduce the computational requirement of attention on such long context, there have been numerous methods proposed to approximate attention [2, 3, 4, 8, 9, 14, 19, 20]. Though these methods have seen some use cases, as far as we know, most large-scale training runs still use standard attention. Motivated by this, Dao et al. [5] proposed to reorder the attention computation and leverages classical techniques (tiling, recomputation) to significantly speed it up and reduce memory usage from quadratic to linear in sequence length. This yields 2-4× wall-clock time speedup over optimized baselines, up to 10-20× memory saving,

[1]FlashAttention-2 is available at https://github.com/Dao-AILab/flash-attention

FlashAttention-2：更快的注意力机制：更好的并行性与工作划分

Tri Dao[1,2]

1普林斯顿大学计算机科学系 [2]斯坦福大学计算机科学系
trid@cs.stanford.edu

2023年7月18日

摘要

在过去的几年中，将Transformer扩展到更长的序列长度一直是一个主要问题，这有望提升语言建模和高分辨率图像理解的性能，并解锁代码、音频和视频生成中的新应用。注意力层是扩展到更长序列的主要瓶颈，因为其运行时间和内存随序列长度呈二次方增长。FlashAttention [5] 利用GPU内存层次结构的不对称性，实现了显著的内存节省（线性而非二次方）和运行速度提升（相比优化基线提升2-4×倍），且无需近似计算。然而，FlashAttention的速度仍远不及优化的矩阵乘法（GEMM）操作，仅达到理论最大FLOPs/s的25-40%。我们发现，这种低效是由于GPU上不同线程块和线程束之间的工作分配不优，导致占用率低或共享内存读写冗余。我们提出了FlashAttention-2，通过改进工作分配来解决这些问题。具体而言，我们（1）调整算法以减少非矩阵乘法FLOPs的数量；（2）将注意力计算（即使是单个头）并行化到不同的线程块，以提高占用率；（3）在每个线程块内，将工作分配给不同线程束，以减少通过共享内存的通信。这些改进使FlashAttention-2相比FlashAttention实现了约2×倍的加速，在A100上达到理论最大FLOPs/s的50-73%，接近GEMM操作的效率。我们通过实证验证，当端到端训练GPT风格模型时，FlashAttention-2在每块A100 GPU上实现了高达225 TFLOPs/s的训练速度（模型FLOPs利用率为72%）。[1]

1 引言

扩展Transformer[18]的上下文长度是一项挑战，因为其核心注意力层的运行时间和内存需求与输入序列长度呈二次方关系。理想情况下，我们希望突破标准的2k序列长度限制，以训练模型理解书籍、高分辨率图像和长视频。仅在去年，就出现了几种上下文长度远超以往的模型：GPT-4[12]支持32k上下文长度，MosaicML的MPT支持65k，Anthropic的Claude则达到100k。长文档查询和故事创作等新兴应用场景，已证明了对这类长上下文模型的迫切需求。

为了降低注意力机制在长上下文上的计算需求，已有大量方法被提出以近似注意力计算[2, 3, 4, 8, 9, 14, 19, 20]。尽管这些方法已在某些场景中得到应用，但据我们所知，大多数大规模训练仍采用标准注意力机制。受此启发，Dao等人[5]提出通过重排注意力计算顺序，并运用经典技术（分块计算、重计算）显著提升计算速度，同时将内存占用从序列长度的二次方降至线性级别。这使得在优化基线上实现了2-4×的实时加速效果，内存节省最高可达10-20×倍。

[1]FlashAttention-2 is available at https://github.com/Dao-AILab/flash-attention

with no approximation, and as a result FlashAttention has seen wide adoption in large-scale training and inference of Transformers.

However, context length increases even more, FlashAttention is still not nearly as efficient as other primitives such as matrix-multiply (GEMM). In particular, while FlashAttention is already 2-4× faster than a standard attention implementation, the forward pass only reaches 30-50% of the theoretical maximum FLOPs/s of the device (Fig. 5), while the backward pass is even more challenging, reaching only 25-35% of maximum throughput on A100 GPU (Fig. 6). In contrast, optimized GEMM can reach up to 80-90% of the theoretical maximum device throughput. Through careful profiling, we observe that FlashAttention still has suboptimal work partitioning between different thread blocks and warps on the GPU, causing either low-occupancy or unnecessary shared memory reads/writes.

Building on FlashAttention, we propose FlashAttention-2 with better parallelism and work partitioning to address these challenges.

1. In Section 3.1, we tweak the algorithms to reduce the number of non-matmul FLOPs while not changing the output. While the non-matmul FLOPs only account for a small fraction of the total FLOPs, they take longer to perform as GPUs have specialized units for matrix multiply, and as a result the matmul throughput can be up to 16× higher than non-matmul throughput. It is thus important to reduce non-matmul FLOPs and spend as much time as possible doing matmul FLOPs.

2. We propose to parallelize both the forward pass and backward pass along the sequence length dimension, in addition to the batch and number of heads dimension. This increases occupancy (utilization of GPU resources) in the case where the sequences are long (and hence batch size is often small).

3. Even within one block of attention computation, we partition the work between different warps of a thread block to reduce communication and shared memory reads/writes.

In Section 4, we empirically validate that FlashAttention-2 yields significant speedup compared to even FlashAttention. Benchmarks on different settings (with or without causal mask, different head dimensions) show that FlashAttention-2 achieves around 2× speedup over FlashAttention, reaching up to 73% of the theoretical max throughput in the forward pass, and up to 63% of the theoretical max throughput in the backward pass. When used end-to-end to train GPT-style models, we reach training speed of up to 225 TFLOPs/s per A100 GPU.

2 Background

We provide some background on the performance characteristics and execution model of GPUs. We also describe the standard implementation of attention, as well as FlashAttention.

2.1 Hardware characteristics

GPU performance characteristics. The GPU consists of compute elements (e.g., floating point arithmetic units) and a memory hierarchy. Most modern GPUs contain specialized units to accelerate matrix multiply in low-precision (e.g., Tensor Cores on Nvidia GPUs for FP16/BF16 matrix multiply). The memory hierarchy comprise of high bandwidth memory (HBM), and on-chip SRAM (aka shared memory). As an example, the A100 GPU has 40-80GB of high bandwidth memory (HBM) with bandwidth 1.5-2.0TB/s and 192KB of on-chip SRAM per each of 108 streaming multiprocessors with bandwidth estimated around 19TB/s [6, 7]. As the L2 cache is not directly controllable by the programmer, we focus on the HBM and SRAM for the purpose of this discussion.

Execution Model. GPUs have a massive number of threads to execute an operation (called a kernel). Threads are organized into thread blocks, which are scheduled to run on streaming multiprocessors (SMs). Within each thread blocks, threads are grouped into warps (a group of 32 threads). Threads within a warp can communicate by fast shuffle instructions or cooperate to perform matrix multiply. Warps within a thread block can communicate by reading from / writing to shared memory. Each kernel loads inputs from HBM to registers and SRAM, computes, then writes outputs to HBM.

无需任何近似处理，因此FlashAttention在Transformer的大规模训练和推理中得到了广泛应用。

然而，随着上下文长度的进一步增加，FlashAttention的效率仍远不及矩阵乘法（GEMM）等其他基础操作。具体而言，尽管FlashAttention已比标准注意力实现快2-4×倍，但其前向传播仅能达到设备理论最大FLOPs/s的30-50%（图5），而后向传播更具挑战性，在A100 GPU上仅能达到最大吞吐量的25-35%（图6）。相比之下，优化后的GEMM可实现设备理论最大吞吐量的80-90%。通过细致分析，我们发现FlashAttention在GPU上不同线程块和线程束之间的任务划分仍非最优，导致计算核心占用率偏低或产生不必要的共享内存读写操作。

基于FlashAttention，我们提出了FlashAttention-2，通过改进并行性和工作划分来应对这些挑战。

1. 在3.1节中，我们调整了算法以减少非矩阵乘法浮点运算的数量，同时不改变输出结果。尽管非矩阵乘法浮点运算仅占总浮点运算的一小部分，但由于GPU拥有专门的矩阵乘法单元，执行这些运算所需时间更长，因此矩阵乘法吞吐量可比非矩阵乘法吞吐量高出高达16×倍。因此，减少非矩阵乘法浮点运算并尽可能将时间用于矩阵乘法浮点运算至关重要。2. 我们提出在批次维度和注意力头数量维度之外，沿序列长度维度并行化前向传播和反向传播过程。这在序列较长（因而批次大小通常较小）的情况下提高了占用率（GPU资源利用率）。3. 即使在单个注意力计算块内部，我们也将工作分配给线程块的不同线程束，以减少通信和共享内存的读写操作。

在第4节中，我们通过实验验证了FlashAttention-2相比FlashAttention能带来显著的加速效果。在不同设置（是否使用因果掩码、不同的头维度）下的基准测试表明，FlashAttention-2相比FlashAttention实现了约2×倍的加速，在前向传播中达到了理论最大吞吐量的73%，在反向传播中达到了理论最大吞吐量的63%。当端到端地用于训练GPT风格模型时，我们在每块A100 GPU上实现了高达225 TFLOPs/s的训练速度。

2 背景

我们提供了一些关于GPU性能特征和执行模型的背景知识。同时，我们也描述了注意力机制的标准实现以及FlashAttention。

2.1 硬件特性

GPU性能特征。GPU由计算单元（例如浮点运算单元）和内存层次结构组成。大多数现代GPU包含专用单元以加速低精度矩阵乘法（例如Nvidia GPU上的Tensor Core用于FP16/BF16矩阵乘法）。内存层次结构包括高带宽内存（HBM）和片上SRAM（又称共享内存）。以A100 GPU为例，其具备40-80GB高带宽内存（HBM），带宽为1.5-2.0TB/s；108个流式多处理器各配备192KB片上SRAM，带宽估计约为19TB/s [6, 7]。由于L2缓存无法由程序员直接控制，本次讨论将重点关注HBM和SRAM。

执行模型。GPU拥有大量线程来执行操作（称为内核）。线程被组织成线程块，这些线程块被调度在流式多处理器（SM）上运行。在每个线程块内，线程被分组为线程束（一组32个线程）。线程束内的线程可以通过快速洗牌指令进行通信，或协作执行矩阵乘法。线程块内的线程束可以通过读取/写入共享内存进行通信。每个内核从HBM加载输入到寄存器和SRAM，进行计算，然后将输出写回HBM。

2.2 Standard Attention Implementation

Given input sequences $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ where N is the sequence length and d is the head dimension, we want to compute the attention output $\mathbf{O} \in \mathbb{R}^{N \times d}$:

$$\mathbf{S} = \mathbf{Q}\mathbf{K}^\top \in \mathbb{R}^{N \times N}, \quad \mathbf{P} = \mathrm{softmax}(\mathbf{S}) \in \mathbb{R}^{N \times N}, \quad \mathbf{O} = \mathbf{P}\mathbf{V} \in \mathbb{R}^{N \times d},$$

where softmax is applied row-wise.[2] For multi-head attention (MHA), this same computation is performed in parallel across many heads, and parallel over the batch dimension (number of input sequences in a batch).

The backward pass of attention proceeds as follows. Let $\mathbf{dO} \in \mathbb{R}^{N \times d}$ be the gradient of $\mathbf{O}$ with respect to some loss function. Then by the chain rule (aka backpropagation):

$$\begin{aligned}
\mathbf{dV} &= \mathbf{P}^\top \mathbf{dO} \in \mathbb{R}^{N \times d} \\
\mathbf{dP} &= \mathbf{dO}\mathbf{V}^\top \in \mathbb{R}^{N \times N} \\
\mathbf{dS} &= \mathrm{dsoftmax}(\mathbf{dP}) \in \mathbb{R}^{N \times N} \\
\mathbf{dQ} &= \mathbf{dS}\mathbf{K} \in \mathbb{R}^{N \times d} \\
\mathbf{dK} &= \mathbf{Q}\mathbf{dS}^\top \in \mathbb{R}^{N \times d},
\end{aligned}$$

where dsoftmax is the gradient (backward pass) of softmax applied row-wise. One can work out that if $p = \mathrm{softmax}(s)$ for some vector s and p, then with output gradient dp, the input gradient $ds = (\mathrm{diag}(p) - pp^\top)dp$.

Standard attention implementations materialize the matrices $\mathbf{S}$ and $\mathbf{P}$ to HBM, which takes $O(N^2)$ memory. Often $N \gg d$ (typically N is on the order of 1k–8k and d is around 64–128). The standard attention implementation (1) calls the matrix multiply (GEMM) subroutine to multiply $\mathbf{S} = \mathbf{Q}\mathbf{K}^\top$, writes the result to HBM, then (2) loads § from HBM to compute softmax and write the result $\mathbf{P}$ to HBM, and finally (3) calls GEMM to get $\mathbf{O} = \mathbf{P}\mathbf{V}$. As most of the operations are bounded by memory bandwidth, the large number of memory accesses translates to slow wall-clock time. Moreover, the required memory is $O(N^2)$ due to having to materialize $\mathbf{S}$ and $\mathbf{P}$. Moreover, one has to save $\mathbf{P} \in \mathbb{R}^{N \times N}$ for the backward pass to compute the gradients.

2.3 FlashAttention

To speed up attention on hardware accelerators such as GPU, [5] proposes an algorithm to reduce the memory reads/writes while maintaining the same output (without approximation).

2.3.1 Forward pass

FlashAttention applies the classical technique of tiling to reduce memory IOs, by (1) loading blocks of inputs from HBM to SRAM, (2) computing attention with respect to that block, and then (3) updating the output without writing the large intermediate matrices $\mathbf{S}$ and $\mathbf{P}$ to HBM. As the softmax couples entire rows or blocks of row, online softmax [11, 13] can split the attention computation into blocks, and rescale the output of each block to finally get the right result (with no approximation). By significantly reducing the amount of memory reads/writes, FlashAttention yields 2-4× wall-clock speedup over optimized baseline attention implementations.

We describe the online softmax technique [11] and how it is used in attention [13]. For simplicity, consider just one row block of the attention matrix $\mathbf{S}$, of the form $\begin{bmatrix}\mathbf{S}^{(1)} & \mathbf{S}^{(2)}\end{bmatrix}$ for some matrices $\mathbf{S}^{(1)}, \mathbf{S}^{(2)} \in \mathbb{R}^{B_r \times B_c}$, where B_r and B_c are the row and column block sizes. We want to compute softmax of this row block and multiply with the value, of the form $\begin{bmatrix}\mathbf{V}^{(1)} \\ \mathbf{V}^{(2)}\end{bmatrix}$ for some matrices $\mathbf{V}^{(1)}, \mathbf{V}^{(2)} \in \mathbb{R}^{B_c \times d}$. Standard softmax would

[2] For clarity of exposition, we omit the scaling of $\mathbf{Q}\mathbf{K}^\top$ (typically by 1/d), and optionally elementwise masking on $\mathbf{S}$ and/or dropout applied to $\mathbf{P}$

2.2 标准注意力机制实现

给定输入序列 $\mathbf{Q}$、$\mathbf{K}$、$\mathbf{V} \in \mathbb{R}^{N \times d}$，其中 为序列长度， 为头维度，我们需要计算注意力输出$\mathbf{O} \in \mathbb{R}^{N \times d}$:

$$\mathbf{S} = \mathbf{Q}\mathbf{K}^\top \in \mathbb{R}^{N \times N}, \quad \mathbf{P} = \mathrm{softmax}(\mathbf{S}) \in \mathbb{R}^{N \times N}, \quad \mathbf{O} = \mathbf{P}\mathbf{V} \in \mathbb{R}^{N \times d},$$

其中softmax按行应用。[2] 对于多头注意力（MHA），这一相同计算会在多个头上并行执行，并在批次维度（一个批次中的输入序列数量）上并行进行。

注意力机制的反向传播过程如下。设 $\mathbf{dO} \in \mathbb{R}^{N \times d}$ 为损失函数对 $\mathbf{O}$ 的梯度，根据链式法则（即反向传播）：

$$\begin{aligned}
\mathbf{dV} &= \mathbf{P}^\top \mathbf{dO} \in \mathbb{R}^{N \times d} \\
\mathbf{dP} &= \mathbf{dO}\mathbf{V}^\top \in \mathbb{R}^{N \times N} \\
\mathbf{dS} &= \mathrm{dsoftmax}(\mathbf{dP}) \in \mathbb{R}^{N \times N} \\
\mathbf{dQ} &= \mathbf{dS}\mathbf{K} \in \mathbb{R}^{N \times d} \\
\mathbf{dK} &= \mathbf{Q}\mathbf{dS}^\top \in \mathbb{R}^{N \times d},
\end{aligned}$$

其中dsoftmax是逐行应用的softmax的梯度（反向传播）。可以推导出，如果对于某个向量 和 存在 softmax( )，那么在输出梯度为 的情况下，输入梯度 = (diag( ) − ⊤) 。

标准的注意力实现将矩阵 $\mathbf{S}$ 和 $\mathbf{P}$ 物化到 HBM 中，这需要 (2) 的内存。通常 ≫ （典型情况下 的数量级为 1k–8k，而 大约为 64–128）。标准的注意力实现 (1) 调用矩阵乘法（GEMM）子程序来相乘 $\mathbf{S} = \mathbf{Q}\mathbf{K}^\top$，将结果写入 HBM，然后 (2) 从 HBM 加载 § 以计算 softmax 并将结果 $\mathbf{P}$ 写入 HBM，最后 (3) 调用 GEMM 来获取 $\mathbf{O} = \mathbf{P}\mathbf{V}$。由于大多数操作受限于内存带宽，大量的内存访问导致实际运行时间变慢。此外，由于必须物化 $\mathbf{S}$ 和 $\mathbf{P}$，所需内存为 (2)。此外，还需要保存 $\mathbf{P} \in \mathbb{R}^{N \times N}$ 用于反向传播以计算梯度。

2.3 FlashAttention

为了在GPU等硬件加速器上加快注意力机制的计算速度，[5]提出了一种算法，能在保持输出不变（无需近似）的同时减少内存读写操作。

2.3.1 前向传播

FlashAttention应用了经典的平铺技术来减少内存I/O，具体通过（1）将输入块从HBM加载到SRAM，（2）针对该块计算注意力，然后（3）更新输出，而无需将大型中间矩阵$\mathbf{S}$和$\mathbf{P}$写入HBM。由于softmax耦合了整个行或行块，在线softmax[11, 13]可以将注意力计算分割成块，并重新缩放每个块的输出，最终获得正确结果（无近似）。通过显著减少内存读写量，FlashAttention在优化的基线注意力实现基础上，实现了2-4×倍的实时加速。

我们描述了在线softmax技术[11]及其在注意力机制中的应用[13]。为简化起见，仅考虑注意力矩阵$\mathbf{S}$的一个行块，其形式为$\begin{bmatrix}\mathbf{S}^{(1)} & \mathbf{S}^{(2)}\end{bmatrix}$，其中$\mathbf{S}^{(1)}$和$\mathbf{S}^{(2)} \in \mathbb{R}^{B_r \times B_c}$为某些矩阵， ，和 $_c$分别为行块和列块的大小。我们需要计算该行块的softmax并与值相乘，其形式为$\begin{bmatrix}\mathbf{V}^{(1)} \\ \mathbf{V}^{(2)}\end{bmatrix}$，其中$\mathbf{V}^{(1)}$和$\mathbf{V}^{(2)} \in \mathbb{R}^{B_c \times d}$为某些矩阵。标准softmax会

[2] For clarity of exposition, we omit the scaling of $\mathbf{Q}\mathbf{K}^\top$ (typically by 1/d), and optionally elementwise masking on $\mathbf{S}$ and/or dropout applied to $\mathbf{P}$

compute:

$$
\begin{aligned}
m &= \max(\text{rowmax}(\mathbf{S}^{(1)}), \text{rowmax}(\mathbf{S}^{(2)})) \in \mathbb{R}^{B_r} \\
\ell &= \text{rowsum}(e^{\mathbf{S}^{(1)}-m}) + \text{rowsum}(e^{\mathbf{S}^{(2)}-m}) \in \mathbb{R}^{B_r} \\
\mathbf{P} &= \begin{bmatrix}\mathbf{P}^{(1)} & \mathbf{P}^{(2)}\end{bmatrix} = \text{diag}(\ell)^{-1}\begin{bmatrix}e^{\mathbf{S}^{(1)}-m} & e^{\mathbf{S}^{(2)}-m}\end{bmatrix} \in \mathbb{R}^{B_r \times 2B_c} \\
\mathbf{O} &= \begin{bmatrix}\mathbf{P}^{(1)} & \mathbf{P}^{(2)}\end{bmatrix}\begin{bmatrix}\mathbf{V}^{(1)} \\ \mathbf{V}^{(2)}\end{bmatrix} = \text{diag}(\ell)^{-1}e^{\mathbf{S}^{(1)}-m}\mathbf{V}^{(1)} + e^{\mathbf{S}^{(2)}-m}\mathbf{V}^{(2)} \in \mathbb{R}^{B_r \times d}.
\end{aligned}
$$

Online softmax instead computes "local" softmax with respect to each block and rescale to get the right output at the end:

$$
\begin{aligned}
m^{(1)} &= \text{rowmax}(\mathbf{S}^{(1)}) \in \mathbb{R}^{B_r} \\
\ell^{(1)} &= \text{rowsum}(e^{\mathbf{S}^{(1)}-m^{(1)}}) \in \mathbb{R}^{B_r} \\
\tilde{\mathbf{P}}^{(1)} &= \text{diag}(\ell^{(1)})^{-1}e^{\mathbf{S}^{(1)}-m^{(1)}} \in \mathbb{R}^{B_r \times B_c} \\
\mathbf{O}^{(1)} &= \tilde{\mathbf{P}}^{(1)}\mathbf{V}^{(1)} = \text{diag}(\ell^{(1)})^{-1}e^{\mathbf{S}^{(1)}-m^{(1)}}\mathbf{V}^{(1)} \in \mathbb{R}^{B_r \times d} \\
m^{(2)} &= \max(m^{(1)}, \text{rowmax}(\mathbf{S}^{(2)})) = m \\
\ell^{(2)} &= e^{m^{(1)}-m^{(2)}}\ell^{(1)} + \text{rowsum}(e^{\mathbf{S}^{(2)}-m^{(2)}}) = \text{rowsum}(e^{\mathbf{S}^{(1)}-m}) + \text{rowsum}(e^{\mathbf{S}^{(2)}-m}) = \ell \\
\tilde{\mathbf{P}}^{(2)} &= \text{diag}(\ell^{(2)})^{-1}e^{\mathbf{S}^{(2)}-m^{(2)}} \\
\mathbf{O}^{(2)} &= \text{diag}(\ell^{(1)}/\ell^{(2)})^{-1}\mathbf{O}^{(1)} + \tilde{\mathbf{P}}^{(2)}\mathbf{V}^{(2)} = \text{diag}(\ell^{(2)})^{-1}e^{s^{(1)}-m}\mathbf{V}^{(1)} + \text{diag}(\ell^{(2)})^{-1}e^{s^{(2)}-m}\mathbf{V}^{(2)} = \mathbf{O}.
\end{aligned}
$$

We show how FlashAttention uses online softmax to enable tiling (Fig. 1) to reduce memory reads/writes.

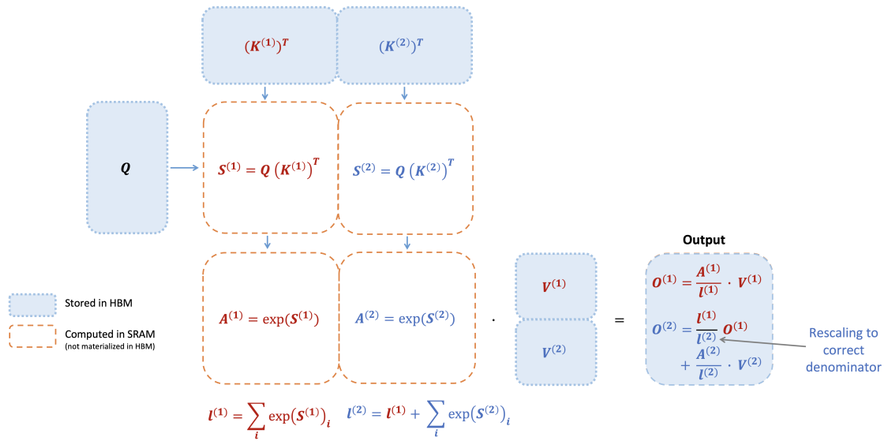


Figure 1: Diagram of how FlashAttention forward pass is performed, when the key $\mathbf{K}$ is partitioned into two blocks and the value $\mathbf{V}$ is also partitioned into two blocks. By computing attention with respect to each block and rescaling the output, we get the right answer at the end, while avoiding expensive memory reads/writes of the intermediate matrices $\mathbf{S}$ and $\mathbf{P}$. We simplify the diagram, omitting the step in softmax that subtracts each element by the row-wise max.

计算：

$$
\begin{aligned}
m &= \max(\text{rowmax}(\mathbf{S}^{(1)}), \text{rowmax}(\mathbf{S}^{(2)})) \in \mathbb{R}^{B_r} \\
\ell &= \text{rowsum}(e^{\mathbf{S}^{(1)}-m}) + \text{rowsum}(e^{\mathbf{S}^{(2)}-m}) \in \mathbb{R}^{B_r} \\
\mathbf{P} &= \begin{bmatrix}\mathbf{P}^{(1)} & \mathbf{P}^{(2)}\end{bmatrix} = \text{diag}(\ell)^{-1}\begin{bmatrix}e^{\mathbf{S}^{(1)}-m} & e^{\mathbf{S}^{(2)}-m}\end{bmatrix} \in \mathbb{R}^{B_r \times 2B_c} \\
\mathbf{O} &= \begin{bmatrix}\mathbf{P}^{(1)} & \mathbf{P}^{(2)}\end{bmatrix}\begin{bmatrix}\mathbf{V}^{(1)} \\ \mathbf{V}^{(2)}\end{bmatrix} = \text{diag}(\ell)^{-1}e^{\mathbf{S}^{(1)}-m}\mathbf{V}^{(1)} + e^{\mathbf{S}^{(2)}-m}\mathbf{V}^{(2)} \in \mathbb{R}^{B_r \times d}.
\end{aligned}
$$

在线softmax则改为对每个块计算“局部”softmax，并在最后重新缩放以获得正确的输出：

$$
\begin{aligned}
m^{(1)} &= \text{rowmax}(\mathbf{S}^{(1)}) \in \mathbb{R}^{B_r} \\
\ell^{(1)} &= \text{rowsum}(e^{\mathbf{S}^{(1)}-m^{(1)}}) \in \mathbb{R}^{B_r} \\
\tilde{\mathbf{P}}^{(1)} &= \text{diag}(\ell^{(1)})^{-1}e^{\mathbf{S}^{(1)}-m^{(1)}} \in \mathbb{R}^{B_r \times B_c} \\
\mathbf{O}^{(1)} &= \tilde{\mathbf{P}}^{(1)}\mathbf{V}^{(1)} = \text{diag}(\ell^{(1)})^{-1}e^{\mathbf{S}^{(1)}-m^{(1)}}\mathbf{V}^{(1)} \in \mathbb{R}^{B_r \times d} \\
m^{(2)} &= \max(m^{(1)}, \text{rowmax}(\mathbf{S}^{(2)})) = m \\
\ell^{(2)} &= e^{m^{(1)}-m^{(2)}}\ell^{(1)} + \text{rowsum}(e^{\mathbf{S}^{(2)}-m^{(2)}}) = \text{rowsum}(e^{\mathbf{S}^{(1)}-m}) + \text{rowsum}(e^{\mathbf{S}^{(2)}-m}) = \ell \\
\tilde{\mathbf{P}}^{(2)} &= \text{diag}(\ell^{(2)})^{-1}e^{\mathbf{S}^{(2)}-m^{(2)}} \\
\mathbf{O}^{(2)} &= \text{diag}(\ell^{(1)}/\ell^{(2)})^{-1}\mathbf{O}^{(1)} + \tilde{\mathbf{P}}^{(2)}\mathbf{V}^{(2)} = \text{diag}(\ell^{(2)})^{-1}e^{s^{(1)}-m}\mathbf{V}^{(1)} + \text{diag}(\ell^{(2)})^{-1}e^{s^{(2)}-m}\mathbf{V}^{(2)} = \mathbf{O}.
\end{aligned}
$$

我们展示了FlashAttention如何利用在线softmax实现分块（图1），以减少内存读写。

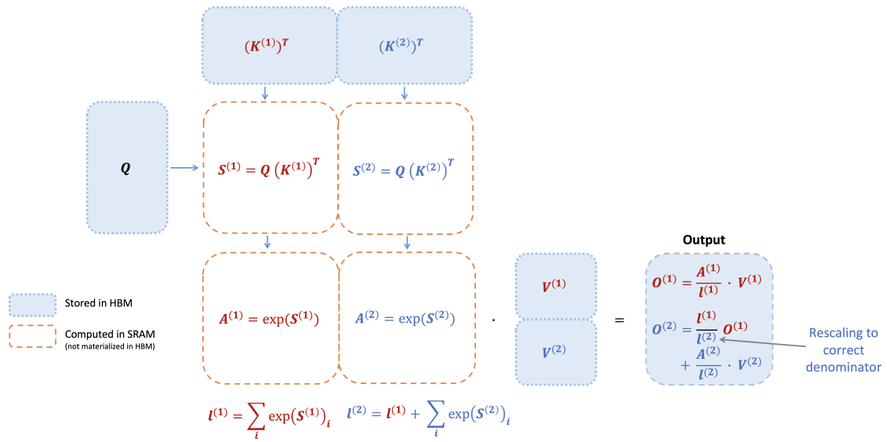


图1：FlashAttention前向传播执行示意图，其中键$\mathbf{K}$被划分为两个块，值$\mathbf{V}$也被划分为两个块。通过对每个块计算注意力并重新缩放输出，我们在最后得到正确结果，同时避免了中间矩阵$\mathbf{S}$和$\mathbf{P}$的昂贵内存读写。我们简化了示意图，省略了softmax中每个元素减去行最大值的步骤。

2.3.2 Backward pass

In the backward pass, by re-computing the values of the attention matrices $\mathbf{S}$ and $\mathbf{P}$ once blocks of inputs $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ are already loaded to SRAM, FlashAttention avoids having to store large intermediate values. By not having to save the large matrices $\mathbf{S}$ and $\mathbf{P}$ of size $N \times N$, FlashAttention yields 10-20× memory saving depending on sequence length (memory required in linear in sequence length N instead of quadratic). The backward pass also achieves 2-4× wall-clock speedup due to reduce memory reads/writes.

The backward pass applies tiling to the equations in Section 2.2. Though the backward pass is simpler than the forward pass conceptually (there is no softmax rescaling), the implementation is significantly more involved. This is because there are more values to be kept in SRAM to perform 5 matrix multiples in the backward pass, compared to just 2 matrix multiples in the forward pass.

3 FlashAttention-2: Algorithm, Parallelism, and Work Partitioning

We describe the FlashAttention-2 algorithm, which includes several tweaks to FlashAttention to reduce the number of non-matmul FLOPs. We then describe how to parallelize the computation on different thread blocks to make full use the GPU resources. Finally we describe we partition the work between different warps within one thread block to reduce the amount of shared memory access. These improvements lead to 2-3× speedup as validated in Section 4.

3.1 Algorithm

We tweak the algorithm from FlashAttention to reduce the number of non-matmul FLOPs. This is because modern GPUs have specialized compute units (e.g., Tensor Cores on Nvidia GPUs) that makes matmul much faster. As an example, the A100 GPU has a max theoretical throughput of 312 TFLOPs/s of FP16/BF16 matmul, but only 19.5 TFLOPs/s of non-matmul FP32. Another way to think about this is that each non-matmul FLOP is 16× more expensive than a matmul FLOP. To maintain high throughput (e.g., more than 50% of the maximum theoretical TFLOPs/s), we want to spend as much time on matmul FLOPs as possible.

3.1.1 Forward pass

We revisit the online softmax trick as shown in Section 2.3 and make two minor tweaks to reduce non-matmul FLOPs:

1. We do not have to rescale both terms of the output update by $\mathrm{diag}(\ell^{(2)})^{-1}$:

$$\mathbf{O}^{(2)} = \mathrm{diag}(\ell^{(1)}/\ell^{(2)})^{-1}\mathbf{O}^{(1)} + \mathrm{diag}(\ell^{(2)})^{-1}e^{\mathbf{S}^{(2)}-m^{(2)}}\mathbf{V}^{(2)}.$$

   We can instead maintain an "un-scaled" version of $\mathbf{O}^{(2)}$ and keep around the statistics $\ell^{(2)}$:

$$\tilde{\mathbf{O}}^{(2)} = \mathrm{diag}(\ell^{(1)})^{-1}\mathbf{O}^{(1)} + e^{\mathbf{S}^{(2)}-m^{(2)}}\mathbf{V}^{(2)}.$$

   Only at the every end of the loop do we scale the final $\tilde{\mathbf{O}}^{(\text{last})}$ by $\mathrm{diag}(\ell^{(\text{last})})^{-1}$ to get the right output.

2. We do not have to save both the max $m^{(j)}$ and the sum of exponentials $\ell^{(j)}$ for the backward pass. We only need to store the logsumexp $L^{(j)} = m^{(j)} + \log(\ell^{(j)})$.

2.3.2 反向传播

在后向传播过程中，通过重新计算注意力矩阵**S**和**P**的值，一旦输入块**Q**、**K**、**V**已加载到SRAM中，FlashAttention就避免了存储大型中间值的需求。由于无需保存大小为 × 的大型矩阵**S**和**P**，FlashAttention根据序列长度实现了10-20×的内存节省（所需内存与序列长度 呈线性关系而非平方关系）。后向传播还因减少了内存读写而实现了2-4×的实际运行速度提升。

反向传播过程对第2.2节中的方程应用了分块处理。虽然从概念上讲，反向传播比前向传播更简单（没有softmax重新缩放），但实现起来却复杂得多。这是因为在反向传播中，需要在SRAM中保存更多的值来执行5次矩阵乘法，而前向传播中只需执行2次矩阵乘法。

3 FlashAttention-2：算法、并行性与工作划分

我们介绍了FlashAttention-2算法，该算法包含对FlashAttention的若干调整，以减少非矩阵乘法浮点运算的次数。接着，我们阐述了如何在不同线程块之间并行化计算，以充分利用GPU资源。最后，我们描述了如何在一个线程块内的不同线程束之间分配工作，以减少共享内存的访问量。这些改进带来了2-3×倍的加速效果，如第4节所验证。

3.1 算法

我们对FlashAttention算法进行了调整，以减少非矩阵乘法浮点运算的次数。这是因为现代GPU拥有专门的计算单元（例如NVIDIA GPU上的张量核心），使得矩阵乘法运算速度大幅提升。以A100 GPU为例，其FP16/BF16矩阵乘法的最大理论吞吐量为312 TFLOPs/s，而非矩阵乘法FP32运算仅为19.5 TFLOPs/s。另一种理解方式是：每个非矩阵乘法浮点运算的成本比矩阵乘法浮点运算高出16×倍。为了保持高吞吐量（例如超过最大理论TFLOPs/s的50%），我们希望尽可能将时间用于矩阵乘法浮点运算。

3.1.1 前向传播

我们重新审视第2.3节中展示的在线softmax技巧，并进行了两处微调以减少非矩阵乘法浮点运算量：

1. 我们无需通过 $\mathrm{diag}(\ell^{(2)})^{-1}$ 对输出更新的两项进行重新缩放：

$$\mathbf{O}^{(2)} = \mathrm{diag}(\ell^{(1)}/\ell^{(2)})^{-1}\mathbf{O}^{(1)} + \mathrm{diag}(\ell^{(2)})^{-1}e^{\mathbf{S}^{(2)}-m^{(2)}}\mathbf{V}^{(2)}.$$

   我们可以转而维护一个未缩放的$\mathbf{O}^{(2)}$版本，并保留统计量$\ell^{(2)}$：

$$\tilde{\mathbf{O}}^{(2)} = \mathrm{diag}(\ell^{(1)})^{-1}\mathbf{O}^{(1)} + e^{\mathbf{S}^{(2)}-m^{(2)}}\mathbf{V}^{(2)}.$$

   只有在循环的最后，我们才将最终的$\tilde{\mathbf{O}}^{(\text{last})}$乘以$\mathrm{diag}(\ell^{(\text{last})})^{-1}$，以获得正确的输出。

2. 我们不必同时保存最大值 $^{(j)}$ 和指数和 $\ell^{(j)}$ 用于反向传播。我们只需要存储对数求和指数 $^{(j)} = {}^{(j)} + \log(\ell^{(j)})$。

In the simple case of 2 blocks in Section 2.3, the online softmax trick now becomes:

$$
\begin{aligned}
m^{(1)} &= \text{rowmax}(\mathbf{S}^{(1)}) \in \mathbb{R}^{B_r} \\
\ell^{(1)} &= \text{rowsum}(e^{\mathbf{S}^{(1)} - m^{(1)}}) \in \mathbb{R}^{B_r} \\
\tilde{\mathbf{O}}^{(1)} &= e^{\mathbf{S}^{(1)} - m^{(1)}} \mathbf{V}^{(1)} \in \mathbb{R}^{B_r \times d} \\
m^{(2)} &= \max(m^{(1)}, \text{rowmax}(\mathbf{S}^{(2)})) = m \\
\ell^{(2)} &= e^{m^{(1)} - m^{(2)}} \ell^{(1)} + \text{rowsum}(e^{\mathbf{S}^{(2)} - m^{(2)}}) = \text{rowsum}(e^{\mathbf{S}^{(1)} - m}) + \text{rowsum}(e^{\mathbf{S}^{(2)} - m}) = \ell \\
\tilde{\mathbf{P}}^{(2)} &= \text{diag}(\ell^{(2)})^{-1} e^{\mathbf{S}^{(2)} - m^{(2)}} \\
\tilde{\mathbf{O}}^{(2)} &= \text{diag}(e^{m^{(1)} - m^{(2)}})^{-1} \tilde{\mathbf{O}}^{(1)} + e^{\mathbf{S}^{(2)} - m^{(2)}} \mathbf{V}^{(2)} = e^{s^{(1)} - m} \mathbf{V}^{(1)} + e^{s^{(2)} - m} \mathbf{V}^{(2)} \\
\mathbf{O}^{(2)} &= \text{diag}(\ell^{(2)})^{-1} \tilde{\mathbf{O}}^{(2)} = \mathbf{O}.
\end{aligned}
$$

We describe the full FlashAttention-2 forward pass in Algorithm 1.

Algorithm 1 FlashAttention-2 forward pass

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, block sizes B_c, B_r.

Divide $\mathbf{Q}$ into $T_r = \left\lceil \frac{N}{B_r} \right\rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ in to $T_c = \left\lceil \frac{N}{B_c} \right\rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
Divide the output $\mathbf{O} \in \mathbb{R}^{N \times d}$ into T_r blocks $\mathbf{O}_i, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, and divide the logsumexp L into T_r blocks $L_i, \dots, L_{T_r}$ of size B_r each.
for $1 \le i \le T_r$ **do**
  Load $\mathbf{Q}_i$ from HBM to on-chip SRAM.
  On chip, initialize $\mathbf{O}_i^{(0)} = (0)_{B_r \times d} \in \mathbb{R}^{B_r \times d}, \ell_i^{(0)} = (0)_{B_r} \in \mathbb{R}^{B_r}, m_i^{(0)} = (-\infty)_{B_r} \in \mathbb{R}^{B_r}$.
  for $1 \le j \le T_c$ **do**
    Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
    On chip, compute $\mathbf{S}_i^{(j)} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
    On chip, compute $m_i^{(j)} = \max(m_i^{(j-1)}, \text{rowmax}(\mathbf{S}_i^{(j)})) \in \mathbb{R}^{B_r}$, $\tilde{\mathbf{P}}_i^{(j)} = \exp(\mathbf{S}_i^{(j)} - m_i^{(j)}) \in \mathbb{R}^{B_r \times B_c}$ (pointwise), $\ell_i^{(j)} = e^{m_i^{j-1} - m_i^{(j)}} \ell_i^{(j-1)} + \text{rowsum}(\tilde{\mathbf{P}}_i^{(j)}) \in \mathbb{R}^{B_r}$.
    On chip, compute $\mathbf{O}_i^{(j)} = \text{diag}(e^{m_i^{(j-1)} - m_i^{(j)}})^{-1} \mathbf{O}_i^{(j-1)} + \tilde{\mathbf{P}}_i^{(j)} \mathbf{V}_j$.
  end for
  On chip, compute $\mathbf{O}_i = \text{diag}(\ell_i^{(T_c)})^{-1} \mathbf{O}_i^{(T_c)}$.
  On chip, compute $L_i = m_i^{(T_c)} + \log(\ell_i^{(T_c)})$.
  Write $\mathbf{O}_i$ to HBM as the i-th block of $\mathbf{O}$.
  Write L_i to HBM as the i-th block of L.
end for
Return the output $\mathbf{O}$ and the logsumexp L.

Causal masking. One common use case of attention is in auto-regressive language modeling, where we need to apply a causal mask to the attention matrix $\mathbf{S}$ (i.e., any entry $\mathbf{S}_{ij}$ with $j > i$ is set to $-\infty$).

1. As FlashAttention and FlashAttention-2 already operate by blocks, for any blocks where all the column indices are more than the row indices (approximately half of the blocks for large sequence length), we can skip the computation of that block. This leads to around 1.7-1.8× speedup compared to attention without the causal mask.

2. We do not need to apply the causal mask for blocks whose row indices are guaranteed to be strictly less than the column indices. This means that for each row, we only need apply causal mask to 1 block (assuming square block).

在2.3节中两个块的简单情况下，在线softmax技巧现在变为：

$$
\begin{aligned}
m^{(1)} &= \text{rowmax}(\mathbf{S}^{(1)}) \in \mathbb{R}^{B_r} \\
\ell^{(1)} &= \text{rowsum}(e^{\mathbf{S}^{(1)} - m^{(1)}}) \in \mathbb{R}^{B_r} \\
\tilde{\mathbf{O}}^{(1)} &= e^{\mathbf{S}^{(1)} - m^{(1)}} \mathbf{V}^{(1)} \in \mathbb{R}^{B_r \times d} \\
m^{(2)} &= \max(m^{(1)}, \text{rowmax}(\mathbf{S}^{(2)})) = m \\
\ell^{(2)} &= e^{m^{(1)} - m^{(2)}} \ell^{(1)} + \text{rowsum}(e^{\mathbf{S}^{(2)} - m^{(2)}}) = \text{rowsum}(e^{\mathbf{S}^{(1)} - m}) + \text{rowsum}(e^{\mathbf{S}^{(2)} - m}) = \ell \\
\tilde{\mathbf{P}}^{(2)} &= \text{diag}(\ell^{(2)})^{-1} e^{\mathbf{S}^{(2)} - m^{(2)}} \\
\tilde{\mathbf{O}}^{(2)} &= \text{diag}(e^{m^{(1)} - m^{(2)}})^{-1} \tilde{\mathbf{O}}^{(1)} + e^{\mathbf{S}^{(2)} - m^{(2)}} \mathbf{V}^{(2)} = e^{s^{(1)} - m} \mathbf{V}^{(1)} + e^{s^{(2)} - m} \mathbf{V}^{(2)} \\
\mathbf{O}^{(2)} &= \text{diag}(\ell^{(2)})^{-1} \tilde{\mathbf{O}}^{(2)} = \mathbf{O}.
\end{aligned}
$$

我们在算法1中描述了完整的FlashAttention-2前向传播过程。

算法1 FlashAttention-2前向传播

要求：矩阵 $\mathbf{Q}$、$\mathbf{K}$、$\mathbf{V} \in \mathbb{R}^{N \times d}$ 位于 HBM 中，块大小为 $_c$、$_r$。 1: 将 $\mathbf{Q}$ 划分为 $_r = \left\lceil \frac{N}{B_r} \right\rceil$ 个块 $\mathbf{Q}_1$、...、$\mathbf{Q}_{T_r}$，每个块大小为 $_r \times$ ；并将$\mathbf{K}$、$\mathbf{V}$ 划分为 $_c = \left\lceil \frac{N}{B_c} \right\rceil$ 个块 $\mathbf{K}_1$、...、$\mathbf{K}_{T_c}$ 和 $\mathbf{V}_1$、...、$\mathbf{V}_{T_c}$，每个块大小为 $_c \times$ 。2: 将输出 $\mathbf{O} \in \mathbb{R}^{N \times d}$ 划分为 $_r$ 个块 $\mathbf{O}_i$、...、$\mathbf{O}_{T_r}$，每个块大小为 $_r \times$ ；并将logsumexp 划分为 $_r$ 个块 $_i$、...、 $_{T_r}$，每个块大小为 $_r$。 3: for $1 \le$ $\le$ $_r$ do 4: 从 HBM 加载 $\mathbf{Q}_i$ 到片上 SRAM。 5: 在片上，初始化 $\mathbf{O}_i^{(0)} = (0)_{B_r \times d} \in \mathbb{R}^{B_r \times d}$、$\ell_i^{(0)} = (0)_{B_r} \in \mathbb{R}^{B_r}$、 $_i^{(0)} = (-\infty)_{B_r} \in \mathbb{R}^{B_r}$。 6: for $1 \le$ $\le$ $_c$ do 7: 从 HBM 加载 $\mathbf{K}_j$、$\mathbf{V}_j$ 到片上 SRAM。 8: 在片上，计算 $\mathbf{S}_i^{(j)} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$。 9: 在片上，计算 $_i^{(j)} = \max($ $_i^{(j-1)}$、$\text{rowmax}(\mathbf{S}_i^{(j)})) \in \mathbb{R}^{B_r}$、$\tilde{\mathbf{P}}_i^{(j)} = \exp(\mathbf{S}_i^{(j)} -$ $_i^{(j)}) \in \mathbb{R}^{B_r \times B_c}$ (逐点)、$\ell_i^{(j)} =$ $m_i^{j-1} - m_i^{(j)}$ $\ell_i^{(j-1)} + \text{rowsum}(\tilde{\mathbf{P}}_i^{(j)}) \in \mathbb{R}^{B_r}$。 10: 在片上，计算 $\mathbf{O}_i^{(j)} = \text{diag}($ $m_i^{(j-1)} - m_i^{(j)})^{-1} \mathbf{O}_i^{(j-1)} + \tilde{\mathbf{P}}_i^{(j)} \mathbf{V}_j$。 11: end for 12: 在片上，计算 $\mathbf{O}_i = \text{diag}(\ell_i^{(T_c)})^{-1} \mathbf{O}_i^{(T_c)}$。 13: 在片上，计算 $_i =$ $_i^{(T_c)} + \log(\ell_i^{(T_c)})$。 14: 将 $\mathbf{O}_i$ 写入 HBM 作为 $\mathbf{O}$ 的第 个块。 15: 将 $_i$ 写入 HBM 作为 的第 个块。 16: end for 17: 返回输出 $\mathbf{O}$ 和 logsumexp 。

因果掩码。注意力机制的一个常见用例是在自回归语言建模中，我们需要对注意力矩阵应用因果掩码$\mathbf{S}$ (，即任何满足 > 的条目$\mathbf{S}_{ij}$都被设置为$-\infty$)。

1. 由于FlashAttention和FlashAttention-2已经按块进行操作，对于所有列索引大于行索引的块（在长序列中约占一半的块），我们可以跳过该块的计算。与不带因果掩码的注意力机制相比，这带来了约1.7-1.8×的加速效果。

2. 对于行索引保证严格小于列索引的块，我们无需应用因果掩码。这意味着对于每一行，我们只需对1个块应用因果掩码（假设为方形块）。

Correctness, runtime, and memory requirement. As with FlashAttention, Algorithm 1 returns the correct output $\mathbf{O} = \text{softmax}(\mathbf{Q}\mathbf{K}^\top)\mathbf{V}$ (with no approximation), using $O(N^2 d)$ FLOPs and requires $O(N)$ additional memory beyond inputs and output (to store the logsumexp L). The proof is almost the same as the proof of Dao et al. [5, Theorem 1], so we omit it here.

3.1.2 Backward pass

The backward pass of FlashAttention-2 is almost the same as that of FlashAttention. We make a minor tweak to only use the row-wise logsumexp L instead of both the row-wise max and row-wise sum of exponentials in the softmax. We include the backward pass description in Algorithm 2 for completeness.

Algorithm 2 FlashAttention-2 Backward Pass

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V}, \mathbf{O}, \mathbf{dO} \in \mathbb{R}^{N \times d}$ in HBM, vector $L \in \mathbb{R}^N$ in HBM, block sizes B_c, B_r.

Divide $\mathbf{Q}$ into $T_r = \left\lceil \frac{N}{B_r} \right\rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ in to $T_c = \left\lceil \frac{N}{B_c} \right\rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
Divide $\mathbf{O}$ into T_r blocks $\mathbf{O}_i, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, divide $\mathbf{dO}$ into T_r blocks $\mathbf{dO}_i, \dots, \mathbf{dO}_{T_r}$ of size $B_r \times d$ each, and divide L into T_r blocks $L_i, \dots, L_{T_r}$ of size B_r each.
Initialize $\mathbf{dQ} = (0)_{N \times d}$ in HBM and divide it into T_r blocks $\mathbf{dQ}_1, \dots, \mathbf{dQ}_{T_r}$ of size $B_r \times d$ each. Divide $\mathbf{dK}, \mathbf{dV} \in \mathbb{R}^{N \times d}$ in to T_c blocks $\mathbf{dK}_1, \dots, \mathbf{dK}_{T_c}$ and $\mathbf{dV}_1, \dots, \mathbf{dV}_{T_c}$, of size $B_c \times d$ each.
Compute $D = \text{rowsum}(\mathbf{dO} \circ \mathbf{O}) \in \mathbb{R}^d$ (pointwise multiply), write D to HBM and divide it into T_r blocks $D_1, \dots, D_{T_r}$ of size B_r each.
for $1 \le j \le T_c$ **do**
  Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
  Initialize $\mathbf{dK}_j = (0)_{B_c \times d}, \mathbf{dV}_j = (0)_{B_c \times d}$ on SRAM.
  for $1 \le i \le T_r$ **do**
    Load $\mathbf{Q}_i, \mathbf{O}_i, \mathbf{dO}_i, \mathbf{dQ}_i, L_i, D_i$ from HBM to on-chip SRAM.
    On chip, compute $\mathbf{S}_i^{(j)} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
    On chip, compute $\mathbf{P}_i^{(j)} = \exp(\mathbf{S}_{ij} - L_i) \in \mathbb{R}^{B_r \times B_c}$.
    On chip, compute $\mathbf{dV}_j \leftarrow \mathbf{dV}_j + (\mathbf{P}_i^{(j)})^\top \mathbf{dO}_i \in \mathbb{R}^{B_c \times d}$.
    On chip, compute $\mathbf{dP}_i^{(j)} = \mathbf{dO}_i \mathbf{V}_j^\top \in \mathbb{R}^{B_r \times B_c}$.
    On chip, compute $\mathbf{dS}_i^{(j)} = \mathbf{P}_i^{(j)} \circ (\mathbf{dP}_i^{(j)} - D_i) \in \mathbb{R}^{B_r \times B_c}$.
    Load $\mathbf{dQ}_i$ from HBM to SRAM, then on chip, update $\mathbf{dQ}_i \leftarrow \mathbf{dQ}_i + \mathbf{dS}_i^{(j)} \mathbf{K}_j \in \mathbb{R}^{B_r \times d}$, and write back to HBM.
    On chip, compute $\mathbf{dK}_j \leftarrow \mathbf{dK}_j + \mathbf{dS}_i^{(j)\top} \mathbf{Q}_i \in \mathbb{R}^{B_c \times d}$.
  end for
  Write $\mathbf{dK}_j, \mathbf{dV}_j$ to HBM.
end for
Return $\mathbf{dQ}, \mathbf{dK}, \mathbf{dV}$.

Multi-query attention and grouped-query attention. Multi-query attention (MQA) [15] and grouped-query attention (GQA) [1] are variants of attention where multiple heads of query attend to the same head of key and value, in order to reduce the size of KV cache during inference. Instead of having to duplicate the key and value heads for the computation, we implicitly manipulate the indices into the head to perform the same computation. In the backward pass, we need to sum the gradients $\mathbf{dK}$ and $\mathbf{dV}$ across different heads that were implicitly duplicated.

3.2 Parallelism

The first version of FlashAttention parallelizes over batch size and number of heads. We use 1 thread block to process one attention head, and there are overall batch size · number of heads thread blocks. Each thread block is scheduled to run on a streaming multiprocessor (SM), and there are 108 of these SMs on

正确性、运行时间和内存需求。与FlashAttention一样，算法1返回正确的输出$\mathbf{O} = \text{softmax}(\mathbf{Q}\mathbf{K}^\top)\mathbf{V}$（，无需近似），使用（^{2}）FLOPs，并且除了输入和输出之外，需要（ ）的额外内存（用于存储logsumexp ）。证明与Dao等人[5，定理1]的证明几乎相同，因此我们在此省略。

3.1.2 反向传播

FlashAttention-2的反向传播过程与FlashAttention几乎相同。我们仅做了一处细微调整：在softmax中仅使用行方向的logsumexp ，而非同时使用行方向的最大值和指数和。为求完整，我们在算法2中包含了反向传播的描述。

Algorithm 2 FlashAttention-2 Backward Pass

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V}, \mathbf{O}, \mathbf{dO} \in \mathbb{R}^{N \times d}$ in HBM, vector $L \in \mathbb{R}^N$ in HBM, block sizes B_c, B_r.

Divide $\mathbf{Q}$ into $T_r = \left\lceil \frac{N}{B_r} \right\rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ in to $T_c = \left\lceil \frac{N}{B_c} \right\rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
Divide $\mathbf{O}$ into T_r blocks $\mathbf{O}_i, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, divide $\mathbf{dO}$ into T_r blocks $\mathbf{dO}_i, \dots, \mathbf{dO}_{T_r}$ of size $B_r \times d$ each, and divide L into T_r blocks $L_i, \dots, L_{T_r}$ of size B_r each.
Initialize $\mathbf{dQ} = (0)_{N \times d}$ in HBM and divide it into T_r blocks $\mathbf{dQ}_1, \dots, \mathbf{dQ}_{T_r}$ of size $B_r \times d$ each. Divide $\mathbf{dK}, \mathbf{dV} \in \mathbb{R}^{N \times d}$ in to T_c blocks $\mathbf{dK}_1, \dots, \mathbf{dK}_{T_c}$ and $\mathbf{dV}_1, \dots, \mathbf{dV}_{T_c}$, of size $B_c \times d$ each.
Compute $D = \text{rowsum}(\mathbf{dO} \circ \mathbf{O}) \in \mathbb{R}^d$ (pointwise multiply), write D to HBM and divide it into T_r blocks $D_1, \dots, D_{T_r}$ of size B_r each.
for $1 \le j \le T_c$ **do**
  Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
  Initialize $\mathbf{dK}_j = (0)_{B_c \times d}, \mathbf{dV}_j = (0)_{B_c \times d}$ on SRAM.
  for $1 \le i \le T_r$ **do**
    Load $\mathbf{Q}_i, \mathbf{O}_i, \mathbf{dO}_i, \mathbf{dQ}_i, L_i, D_i$ from HBM to on-chip SRAM.
    On chip, compute $\mathbf{S}_i^{(j)} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
    On chip, compute $\mathbf{P}_i^{(j)} = \exp(\mathbf{S}_{ij} - L_i) \in \mathbb{R}^{B_r \times B_c}$.
    On chip, compute $\mathbf{dV}_j \leftarrow \mathbf{dV}_j + (\mathbf{P}_i^{(j)})^\top \mathbf{dO}_i \in \mathbb{R}^{B_c \times d}$.
    On chip, compute $\mathbf{dP}_i^{(j)} = \mathbf{dO}_i \mathbf{V}_j^\top \in \mathbb{R}^{B_r \times B_c}$.
    On chip, compute $\mathbf{dS}_i^{(j)} = \mathbf{P}_i^{(j)} \circ (\mathbf{dP}_i^{(j)} - D_i) \in \mathbb{R}^{B_r \times B_c}$.
    Load $\mathbf{dQ}_i$ from HBM to SRAM, then on chip, update $\mathbf{dQ}_i \leftarrow \mathbf{dQ}_i + \mathbf{dS}_i^{(j)} \mathbf{K}_j \in \mathbb{R}^{B_r \times d}$, and write back to HBM.
    On chip, compute $\mathbf{dK}_j \leftarrow \mathbf{dK}_j + \mathbf{dS}_i^{(j)\top} \mathbf{Q}_i \in \mathbb{R}^{B_c \times d}$.
  end for
  Write $\mathbf{dK}_j, \mathbf{dV}_j$ to HBM.
end for
Return $\mathbf{dQ}, \mathbf{dK}, \mathbf{dV}$.

多头查询注意力与分组查询注意力。多头查询注意力（MQA）[15]和分组查询注意力（GQA）[1]是注意力机制的变体，其中多个查询头关注同一个键和值头，以减少推理过程中KV缓存的大小。我们通过隐式操作头索引来执行相同的计算，而无需为计算复制键和值头。在反向传播过程中，我们需要对不同头之间隐式复制的梯度$\mathbf{dK}$和$\mathbf{dV}$进行求和。

3.2 并行性

FlashAttention的第一个版本在批大小和注意力头数量上并行化。我们使用1个线程块处理一个注意力头，总共有批大小·乘以注意力头数量的线程块。每个线程块被调度在流式多处理器（SM）上运行，而总共有108个这样的SM。

an A100 GPU for example. This scheduling is efficient when this number is large (say ≥ 80), since we can effectively use almost all of the compute resources on the GPU.

In the case of long sequences (which usually means small batch sizes or small number of heads), to make better use of the multiprocessors on the GPU, we now additionally parallelize over the sequence length dimension. This results in significant speedup for this regime.

Forward pass. We see that the outer loop (over sequence length) is embarrassingly parallel, and we schedule them on different thread blocks that do not need to communicate with each other. We also parallelize over the batch dimension and number of heads dimension, as done in FlashAttention. The increased parallelism over sequence length helps improve occupancy (fraction of GPU resources being used) when the batch size and number of heads are small, leading to speedup in this case.

These ideas of swapping the order of the loop (outer loop over row blocks and inner loop over column blocks, instead of the other way round in the original FlashAttention paper), as well as parallelizing over the sequence length dimension were first suggested and implemented by Phil Tillet in the Triton [17] implementation.[3]

Backward pass. Notice that the only shared computation between different column blocks is in update $\mathbf{dQ}$ in Algorithm 2, where we need to load $\mathbf{dQ}_i$ from HBM to SRAM, then on chip, update $\mathbf{dQ}_i \leftarrow \mathbf{dQ}_i + \mathbf{dS}_i^{(j)}\mathbf{K}_j$, and write back to HBM. We thus parallelize over the sequence length dimension as well, and schedule 1 thread block for each column block of the backward pass. We use atomic adds to communicate between different thread blocks to update $\mathbf{dQ}$.

We describe the parallelization scheme in Fig. 2.

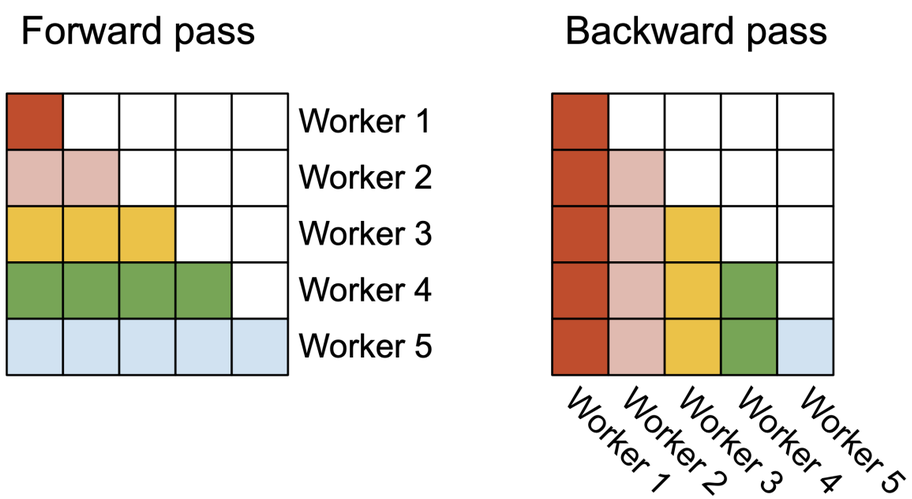


Figure 2: In the forward pass (left), we parallelize the workers (thread blocks) where each worker takes care of a block of rows of the attention matrix. In the backward pass (right), each worker takes care of a block of columns of the attention matrix.

[3] https://github.com/openai/triton/blob/main/python/tutorials/06-fused-attention.py

以A100 GPU为例。当这个数字很大时（比如≥ 80），这种调度方式是高效的，因为我们可以有效地利用GPU上几乎所有的计算资源。

在处理长序列时（通常意味着较小的批量大小或较少的注意力头数），为了更有效地利用GPU上的多处理器，我们现在额外在序列长度维度上进行并行化。这为该场景带来了显著的加速效果。

前向传播。我们看到，外层循环（遍历序列长度）具有令人尴尬的并行性，我们将它们调度到无需相互通信的不同线程块上。我们还沿批次维度和注意力头数量维度进行并行化，正如FlashAttention中所做的那样。当批次大小和注意力头数量较小时，沿序列长度增加的并行性有助于提高占用率（GPU资源使用比例），从而在这种情况下实现加速。

这些关于交换循环顺序（外循环遍历行块，内循环遍历列块，而非原FlashAttention论文中的相反方式）以及沿序列长度维度并行化的想法，最初由Phil Tillet在Triton[17]实现中提出并实施。[3]

反向传播过程。注意不同列块之间唯一的共享计算是在算法2中更新$\mathbf{dQ}$时，我们需要将$\mathbf{dQ}_i$从HBM加载到SRAM，然后在芯片上更新$\mathbf{dQ}_i \leftarrow \mathbf{dQ}_i + \mathbf{dS}_i^{(j)}\mathbf{K}_j$，并写回HBM。因此，我们同样沿序列长度维度进行并行化，并为反向传播的每个列块调度1个线程块。我们使用原子加法在不同线程块之间通信以更新$\mathbf{dQ}$。

我们在图2中描述了并行化方案。

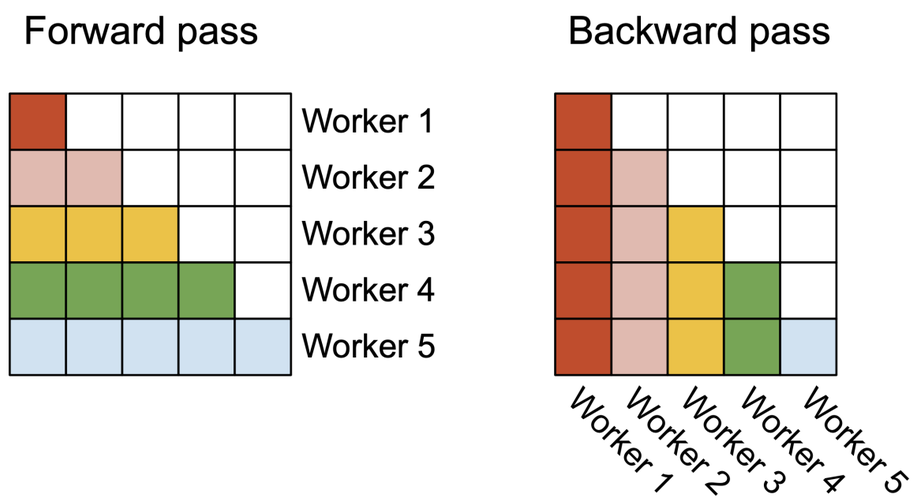


图2：在前向传播（左）中，我们并行化处理工作单元（线程块），每个工作单元负责注意力矩阵的一个行块。在后向传播（右）中，每个工作单元负责注意力矩阵的一个列块。

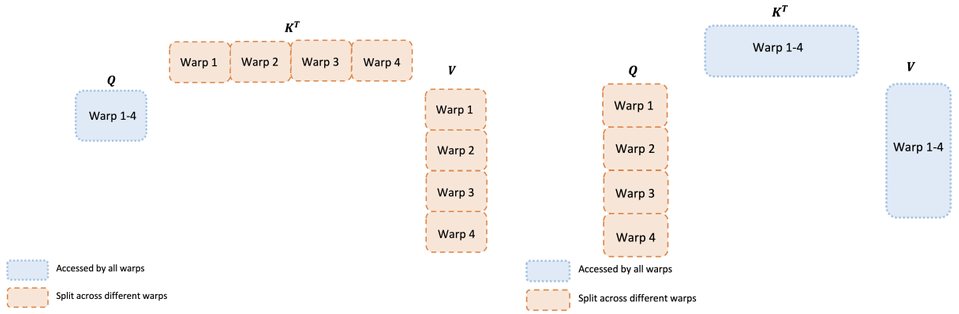
[3] https://github.com/openai/triton/blob/main/python/tutorials/06-fused-attention.py

3.3 Work Partitioning Between Warps

As Section 3.2 describe how we schedule thread blocks, even within each thread block, we also have to decide how to partition the work between different warps. We typically use 4 or 8 warps per thread block, and the partitioning is described in Fig. 3.

Forward pass. For each block, FlashAttention splits $\mathbf{K}$ and $\mathbf{V}$ across 4 warps while keeping $\mathbf{Q}$ accessible by all warps. Each warp multiplies to get a slice of $\mathbf{QK}^\top$, then they need to multiply with a slice of $\mathbf{V}$ and communicate to add up the result. This is referred to as the "split-K" scheme. However, this is inefficient since all warps need to write their intermediate results out to shared memory, synchronize, then add up the intermediate results. These shared memory reads/writes slow down the forward pass in FlashAttention.

In FlashAttention-2, we instead split $\mathbf{Q}$ across 4 warps while keeping $\mathbf{K}$ and $\mathbf{V}$ accessible by all warps. After each warp performs matrix multiply to get a slice of $\mathbf{QK}^\top$, they just need to multiply with their shared slice of $\mathbf{V}$ to get their corresponding slice of the output. There is no need for communication between warps. The reduction in shared memory reads/writes yields speedup (Section 4).



(a) FlashAttention (b) FlashAttention-2

Figure 3: Work partitioning between different warps in the forward pass

Backward pass. Similarly for the backward pass, we choose to partition the warps to avoid the "split-K" scheme. However, it still requires some synchronization due to the more complicated dependency between all the different inputs and gradients $\mathbf{Q}, \mathbf{K}, \mathbf{V}, \mathbf{O}, \mathbf{dO}, \mathbf{dQ}, \mathbf{dK}, \mathbf{dV}$. Nevertheless, avoiding "split-K" reduces shared memory reads/writes and again yields speedup (Section 4).

Tuning block sizes Increasing block sizes generally reduces shared memory loads/stores, but increases the number of registers required and the total amount of shared memory. Past a certain block size, register spilling causes significant slowdown, or the amount of shared memory required is larger than what the GPU has available, and the kernel cannot run at all. Typically we choose blocks of size $\{64, 128\} \times \{64, 128\}$, depending on the head dimension d and the device shared memory size.

We manually tune for each head dimensions since there are essentially only 4 choices for block sizes, but this could benefit from auto-tuning to avoid this manual labor. We leave this to future work.

4 Empirical Validation

We evaluate the impact of using FlashAttention-2 to train Transformer models.

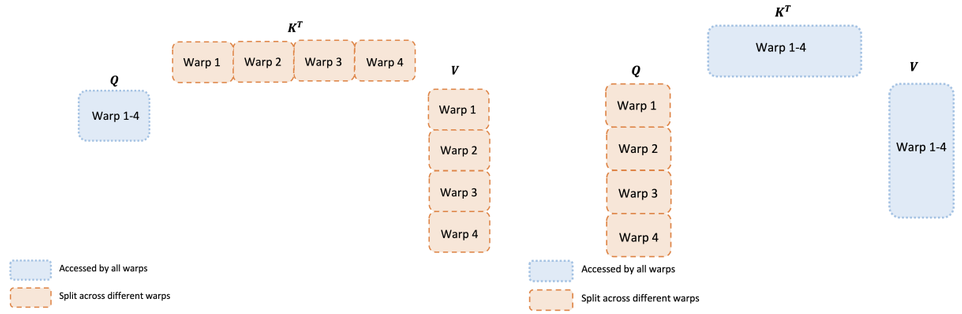
- **Benchmarking attention.** We measure the runtime of FlashAttention-2 across different sequence lengths and compare it to a standard implementation in PyTorch, FlashAttention, and FlashAttention in Triton. We confirm that FlashAttention-2 is 1.7-3.0× faster than FlashAttention, 1.3-2.5× faster than FlashAttention in Triton, and 3-10× faster than a standard attention implementation.

3.3 线程束间的工作划分

如第3.2节所述，我们如何调度线程块，即便在每个线程块内部，我们也必须决定如何在不同线程束之间分配工作。我们通常每个线程块使用4或8个线程束，具体分配方式如图3所示。

前向传播。对于每个块，FlashAttention将$\mathbf{K}$和$\mathbf{V}$分割到4个线程束中，同时保持$\mathbf{Q}$对所有线程束可访问。每个线程束进行乘法运算得到$\mathbf{QK}^\top$的一个切片，然后需要与$\mathbf{V}$的一个切片相乘并进行通信以累加结果。这被称为"分割K"方案。然而，这种做法效率低下，因为所有线程束都需要将中间结果写入共享内存，同步后再累加中间结果。这些共享内存的读写操作降低了FlashAttention中前向传播的速度。

在FlashAttention-2中，我们将$\mathbf{Q}$分割到4个线程束中，同时保持$\mathbf{K}$和$\mathbf{V}$对所有线程束可访问。每个线程束执行矩阵乘法得到$\mathbf{QK}^\top$的一个切片后，只需与它们共享的$\mathbf{V}$切片相乘，即可得到对应的输出切片。线程束之间无需通信。共享内存读写操作的减少带来了加速效果（第4节）。



(a) FlashAttention (b) FlashAttention-2

图3：前向传播中不同线程束之间的工作划分

反向传播。类似地，对于反向传播，我们选择对线程束进行分区以避免"split-K"方案。然而，由于所有不同输入和梯度$\mathbf{Q}$、$\mathbf{K}$、$\mathbf{V}$、$\mathbf{O}$、$\mathbf{dO}$、$\mathbf{dQ}$、$\mathbf{dK}$、$\mathbf{dV}$之间更复杂的依赖关系，仍然需要一些同步。尽管如此，避免"split-K"减少了共享内存的读写操作，并再次实现了加速（第4节）。

调整块大小　增加块大小通常会减少共享内存的加载/存储操作，但会增加所需的寄存器数量和共享内存总量。超过特定块大小时，寄存器溢出会导致显著减速，或者所需的共享内存量超过GPU可用容量，内核将完全无法运行。通常我们根据头维度　和设备共享内存大小，选择大小为$\{64, 128\} \times \{64, 128\}$的块。

由于块大小本质上只有4种选择，我们手动调整每个头的维度，但这可以通过自动调优来避免这种手动劳动。我们将这一点留给未来的工作。

4 实证验证

我们评估了使用FlashAttention-2训练Transformer模型的影响。

- 基准测试注意力机制。我们测量了FlashAttention-2在不同序列长度下的运行时间，并将其与PyTorch中的标准实现、FlashAttention以及Triton中的FlashAttention进行了比较。我们确认FlashAttention-2比FlashAttention快1.7-3.0×倍，比Triton中的FlashAttention快1.3-2.5×倍，比标准注意力实现快3-10×倍。

FlashAttention-2 reaches up to 230 TFLOPs/s, 73% of the theoretical maximum TFLOPs/s on A100 GPUs.

- **End-to-end training speed** When used end-to-end to train GPT-style models of size 1.3B and 2.7B on sequence lengths either 2k or 8k, FlashAttention-2 yields up to 1.3× speedup compared to FlashAttention and 2.8× speedup compared to a baseline without FlashAttention. FlashAttention-2 reaches up to 225 TFLOPs/s (72% model FLOPs utilization) per A100 GPU.

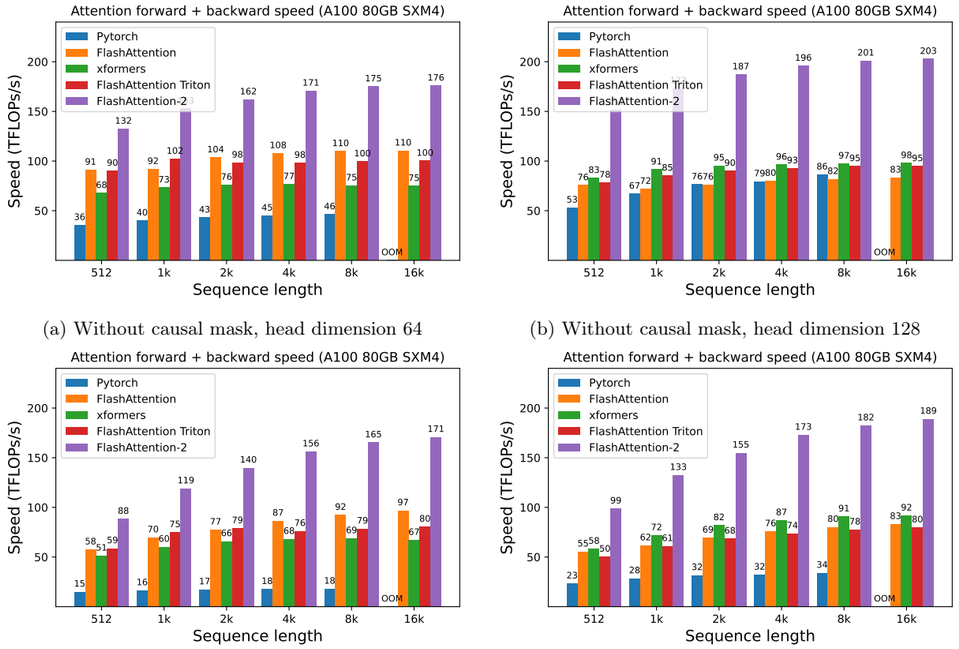
4.1 Benchmarking Attention

We measure the runtime of different attention methods on an A100 80GB SXM4 GPU for different settings (without / with causal mask, head dimension 64 or 128). We report the results in Fig. 4, Fig. 5 and Fig. 6, showing that FlashAttention-2 is around 2× faster than FlashAttention and FlashAttention in `xformers` (the "cutlass" implementation). FlashAttention-2 is around 1.3-1.5× faster than FlashAttention in Triton in the forward pass and around 2× faster in the backward pass. Compared to a standard attention implementation in PyTorch, FlashAttention-2 can be up to 10× faster.

Benchmark setting: we vary the sequence length from 512, 1k, ..., 16k, and set batch size so that the total number of tokens is 16k. We set hidden dimension to 2048, and head dimension to be either 64 or 128 (i.e., 32 heads or 16 heads). To calculate the FLOPs of the forward pass, we use:

$$4 \cdot \text{seqlen}^2 \cdot \text{head dimension} \cdot \text{number of heads}.$$

With causal mask, we divide this number by 2 to account for the fact that approximately only half of the entries are calculated. To get the FLOPs of the backward pass, we multiply the forward pass FLOPs by 2.5 (since there are 2 matmuls in the forward pass and 5 matmuls in the backward pass, due to recomputation).



(a) Without causal mask, head dimension 64

(b) Without causal mask, head dimension 128

(c) With causal mask, head dimension 64

(d) With causal mask, head dimension 128

Figure 4: Attention forward + backward speed on A100 GPU

FlashAttention-2 在 A100 GPU 上实现了高达 230 TFLOPs/s 的运算速度，达到理论最大 TFLOPs/s 的 73%。

- 端到端训练速度　当使用端到端方式训练规模为1亿和27亿、序列长度为2k或8k的GPT风格模型时，FlashAttention-2相比FlashAttention最高可带来1.3×倍加速，相比未使用FlashAttention的基线方案最高可提升2.8×倍速度。FlashAttention-2在每张A100 GPU上最高可实现225 TFLOPs/s（72%的模型浮点运算利用率）。

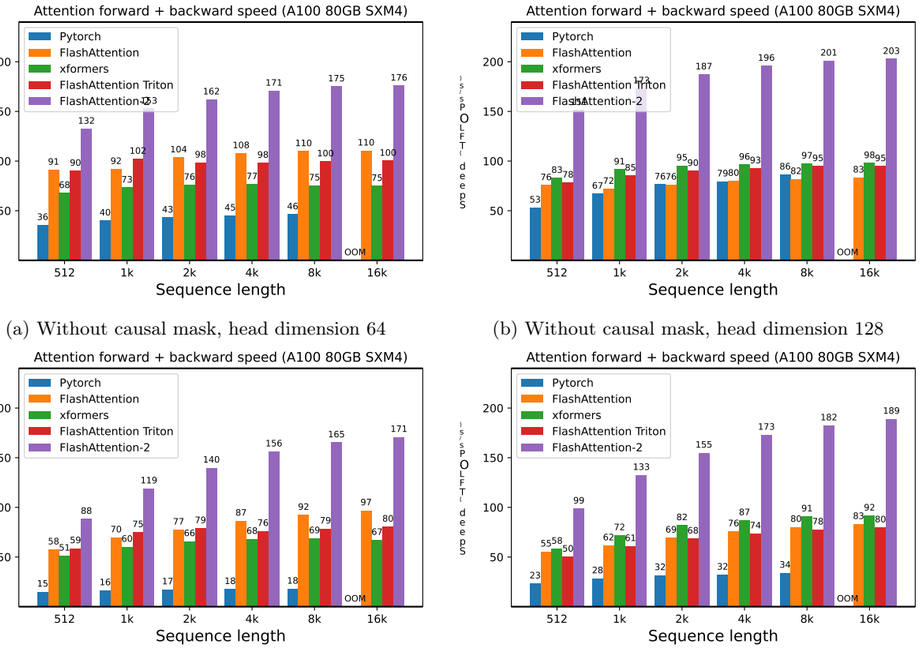
4.1 注意力机制基准测试

我们在A100 80GB SXM4 GPU上测量了不同设置下（无/有因果掩码，头维度64或128）各种注意力方法的运行时间。结果如图4、图5和图6所示，表明FlashAttention-2比FlashAttention及xformers中的FlashAttention（“cutlass”实现）快约2×倍。在前向传播中，FlashAttention-2比Triton中的FlashAttention快约1.3-1.5×倍，在后向传播中快约2×倍。与PyTorch中的标准注意力实现相比，FlashAttention-2最高可快10×倍。

基准设置：我们将序列长度从512、1k、...、16k进行变化，并设置批次大小以使总标记数为16k。我们将隐藏维度设置为2048，头部维度设置为64或128（即32个头或16个头）。为了计算前向传播的FLOPs，我们使用：

$$4 \cdot \text{序列长度}^2 \cdot \text{头部维度} \cdot \text{头部数量}。$$

使用因果掩码时，我们将这个数字除以2，以考虑大约只有一半的条目被计算的事实。为了获得反向传播的FLOPs，我们将前向传播的FLOPs乘以2.5（因为前向传播中有2次矩阵乘法，而反向传播中有5次矩阵乘法，这是由于重计算导致的）。



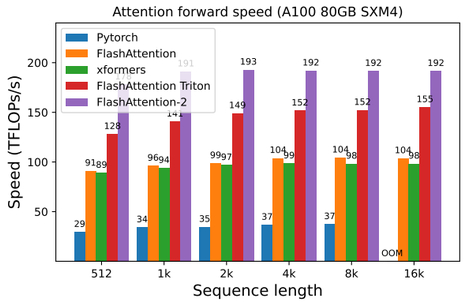
(a) Without causal mask, head dimension 64

(b) Without causal mask, head dimension 128

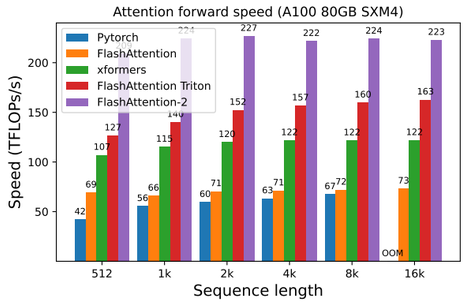
(c) With causal mask, head dimension 64

(d) With causal mask, head dimension 128

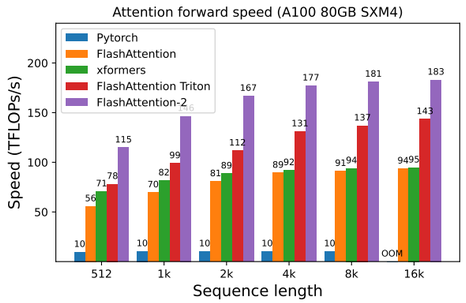
图4：A100 GPU上的注意力前向+后向速度



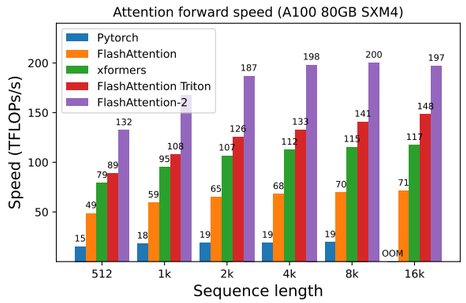
(a) Without causal mask, head dimension 64



(b) Without causal mask, head dimension 128



(c) With causal mask, head dimension 64



(d) With causal mask, head dimension 128

Figure 5: Attention forward speed on A100 GPU

Just running the same implementation on H100 GPUs (using no special instructions to make use of new features such as TMA and 4th-gen Tensor Cores), we obtain up to 335 TFLOPs/s (Fig. 7). We expect that by using new instructions, we can obtain another 1.5x-2x speedup on H100 GPUs. We leave that to future work.

4.2 End-to-end Performance

We measure the training throughput of GPT-style models with either 1.3B or 2.7B parameters, on 8×A100 80GB SXM. As shown in Table 1, FlashAttention-2 yields 2.8× speedup compared to a baseline without FlashAttention and 1.3× speedup compared to FlashAttention-2, reaching up to 225 TFLOPs/s per A100 GPU.

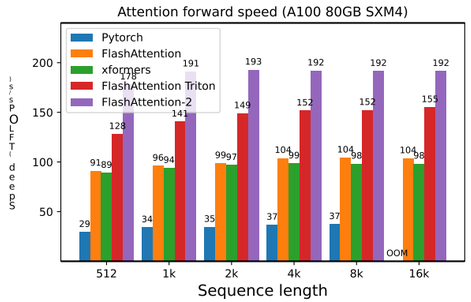
Note that we calculate the FLOPs by the formula, following Megatron-LM [16] (and many other papers and libraries):

$$6 \cdot \text{seqlen} \cdot \text{number of params} + 12 \cdot \text{number of layers} \cdot \text{hidden dim} \cdot \text{seqlen}^2.$$

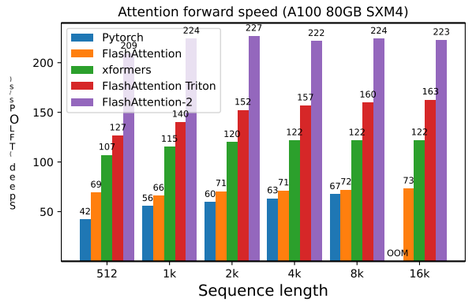
The first term accounts for the FLOPs due to weight–input multiplication, and the second term accounts for the FLOPs due to attention. However, one can argue that the second term should be halved, as with causal mask we only need to compute approximately half the number of elements in attention. We choose to follow the formula from the literature (without dividing the attention FLOPs by 2) for consistency.

5 Discussion and Future Directions

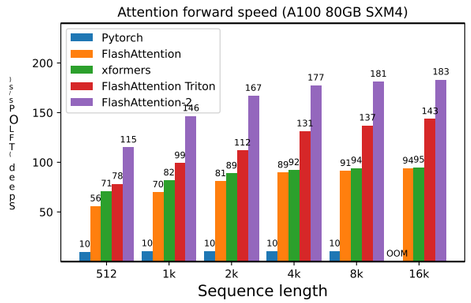
FlashAttention-2 is 2× faster than FlashAttention, which means that we can train models with 16k longer context for the same price as previously training a 8k context model. We are excited about how this can



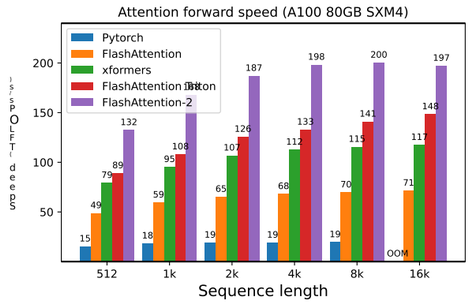
(a) Without causal mask, head dimension 64



(b) Without causal mask, head dimension 128



(c) With causal mask, head dimension 64



(d) With causal mask, head dimension 128

图5：A100 GPU上的注意力前向速度

仅在同一H100 GPU上运行相同的实现（未使用特殊指令来利用TMA和第四代Tensor Cores等新功能），我们就能获得高达335 TFLOPs/s的性能（图7）。我们预计，通过使用新指令，可以在H100 GPU上再获得1.5倍至2倍的加速。我们将此留待未来研究。

4.2 端到端性能

我们在8块×A100 80GB SXM GPU上，对参数规模为13亿或27亿的GPT风格模型进行了训练吞吐量测试。如表1所示，FlashAttention-2相比未使用FlashAttention的基线实现了2.8×倍加速，相比FlashAttention-1则获得1.3×倍加速，使每块A100 GPU的算力最高达到225 TFLOPs/s。

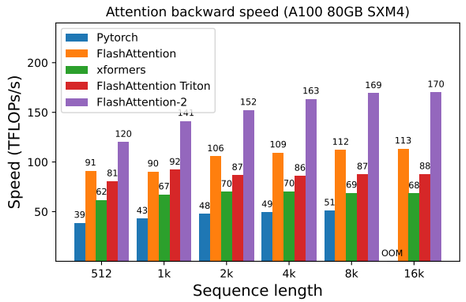
请注意，我们按照Megatron-LM [16]（以及许多其他论文和库）的公式计算FLOPs：

$$6 \cdot \text{序列长度参数数量} + 12 \cdot \text{层数} \cdot \text{隐藏维度} \cdot \text{s} \qquad \text{eqlen}^2\text{。}$$

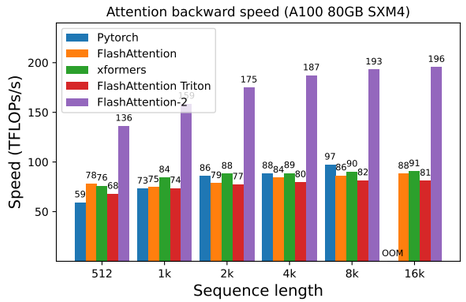
第一项考虑了权重与输入相乘所产生的浮点运算次数，第二项则考虑了注意力机制所产生的浮点运算次数。然而，有人可能会认为第二项应该减半，因为在因果掩码下，我们只需要计算注意力中大约一半的元素。为了保持一致性，我们选择遵循文献中的公式（不将注意力浮点运算次数除以2）。

5 讨论与未来方向

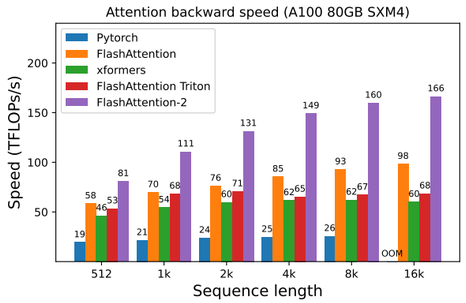
FlashAttention-2 比 FlashAttention 快 2× 倍，这意味着我们可以用之前训练 8k 上下文模型的同等成本，训练出上下文长度增加 16k 的模型。我们对这一进展所能带来的可能性感到振奋。



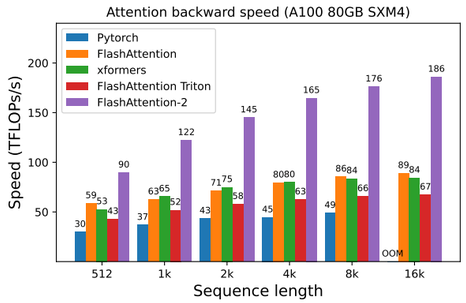
(a) Without causal mask, head dimension 64



(b) Without causal mask, head dimension 128



(c) With causal mask, head dimension 64



(d) With causal mask, head dimension 128

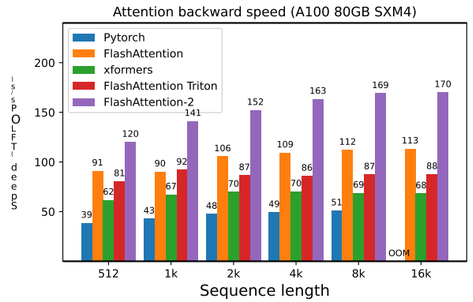
Figure 6: Attention backward speed on A100 GPU

Table 1: Training speed (TFLOPs/s/GPU) of GPT-style models on 8×A100 GPUs. FlashAttention-2 reaches up to 225 TFLOPs/s (72% model FLOPs utilization). We compare against a baseline running without FlashAttention.

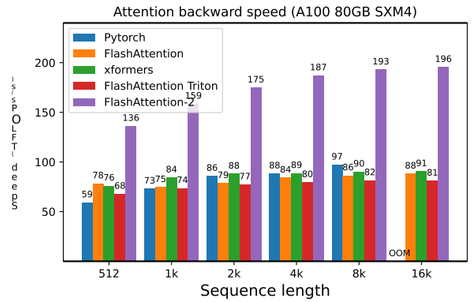
Model	Without FlashAttention	FlashAttention	FlashAttention-2
GPT3-1.3B 2k context	142 TFLOPs/s	189 TFLOPs/s	196 TFLOPs/s
GPT3-1.3B 8k context	72 TFLOPS/s	170 TFLOPs/s	220 TFLOPs/s
GPT3-2.7B 2k context	149 TFLOPs/s	189 TFLOPs/s	205 TFLOPs/s
GPT3-2.7B 8k context	80 TFLOPs/s	175 TFLOPs/s	225 TFLOPs/s

be used to understand long books and reports, high resolution images, audio and video. FlashAttention-2 will also speed up training, finetuning, and inference of existing models.

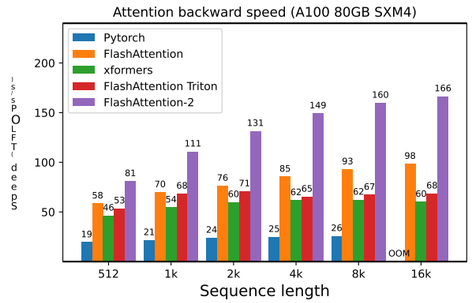
In the near future, we plan to collaborate with researchers and engineers to make FlashAttention widely applicable in different kinds of devices (e.g., H100 GPUs, AMD GPUs), as well as new data types such as FP8. As an immediate next step, we plan to optimize FlashAttention-2 for H100 GPUs to use new hardware features (TMA, 4th-gen Tensor Cores, fp8). Combining the low-level optimizations in FlashAttention-2 with high-level algorithmic changes (e.g., local, dilated, block-sparse attention) could allow us to train AI models with much longer context. We are also excited to work with compiler researchers to make these optimization techniques easily programmable.



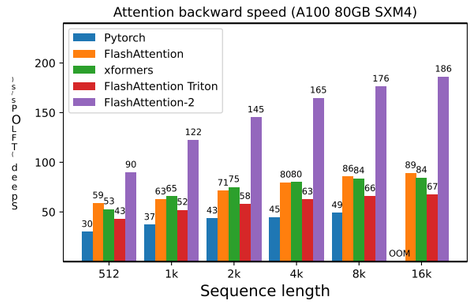
(a) Without causal mask, head dimension 64



(b) Without causal mask, head dimension 128



(c) With causal mask, head dimension 64



(d) With causal mask, head dimension 128

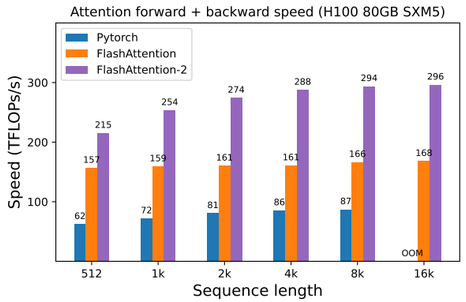
图6：A100 GPU上的注意力反向传播速度

表1：在8×A100 GPU上GPT风格模型的训练速度（TFLOPs/s/GPU）。FlashAttention-2最高可达225 TFLOPs/s（72%模型FLOPs利用率）。我们对比了未使用FlashAttention的基线运行结果。

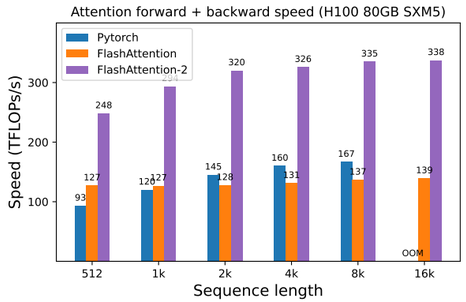
Model	Without FlashAttention	FlashAttention	FlashAttention-2
GPT3-1.3B 2k context	142 TFLOPs/s	189 TFLOPs/s	196 TFLOPs/s
GPT3-1.3B 8k context	72 TFLOPS/s	170 TFLOPs/s	220 TFLOPs/s
GPT3-2.7B 2k context	149 TFLOPs/s	189 TFLOPs/s	205 TFLOPs/s
GPT3-2.7B 8k context	80 TFLOPs/s	175 TFLOPs/s	225 TFLOPs/s

用于理解长篇书籍和报告、高分辨率图像、音频和视频。FlashAttention-2还将加速现有模型的训练、微调和推理。

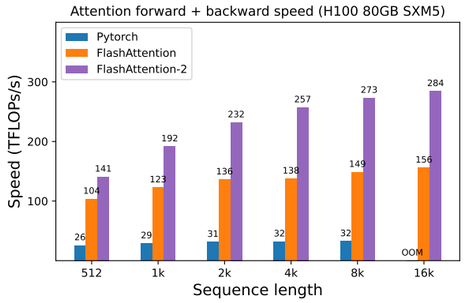
在不久的将来，我们计划与研究人员和工程师合作，使FlashAttention能够广泛应用于不同类型的设备（例如H100 GPU、AMD GPU）以及新的数据类型（如FP8）。作为接下来的直接步骤，我们计划针对H100 GPU优化FlashAttention-2，以利用新的硬件特性（TMA、第四代Tensor Core、fp8）。将FlashAttention-2中的底层优化与高层算法变更（例如局部、扩张、块稀疏注意力）相结合，将使我们能够训练具有更长上下文的AI模型。我们也期待与编译器研究人员合作，使这些优化技术更易于编程实现。



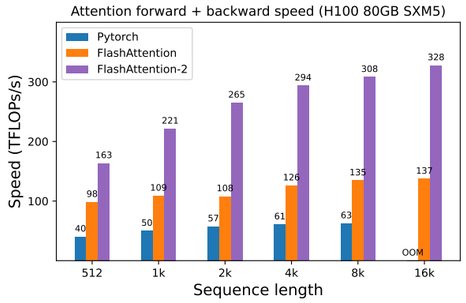
(a) Without causal mask, head dimension 64



(b) Without causal mask, head dimension 128



(c) With causal mask, head dimension 64



(d) With causal mask, head dimension 128

Figure 7: Attention forward + backward speed on H100 GPU

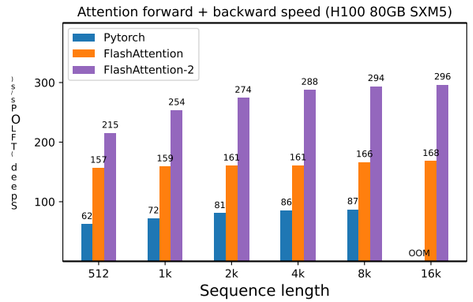
Acknowledgments

We thank Phil Tillet and Daniel Haziza, who have implemented versions of FlashAttention in Triton [17] and the `xformers` library [10]. FlashAttention-2 was motivated by exchange of ideas between different ways that attention could be implemented. We are grateful to the Nvidia CUTLASS team (especially Vijay Thakkar, Cris Cecka, Haicheng Wu, and Andrew Kerr) for their CUTLASS library, in particular the CUTLASS 3.x release, which provides clean abstractions and powerful building blocks for the implementation of FlashAttention-2. We thank Driss Guessous for integrating FlashAttention to PyTorch. FlashAttention-2 has benefited from helpful discussions with Phil Wang, Markus Rabe, James Bradbury, Young-Jun Ko, Julien Launay, Daniel Hesslow, Michaël Benesty, Horace He, Ashish Vaswani, and Erich Elsen. Thanks for Stanford CRFM and Stanford NLP for the compute support. We thank Dan Fu and Christopher Ré for their collaboration, constructive feedback, and constant encouragement on this line of work of designing hardware-efficient algorithms. We thank Albert Gu and Beidi Chen for their helpful suggestions on early drafts of this technical report.

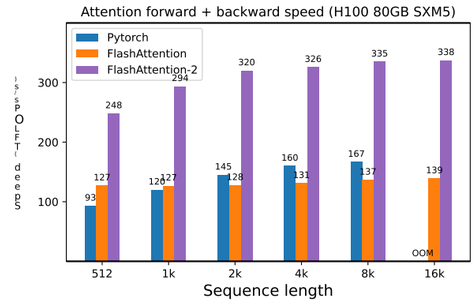
References

[1] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. Gqa: Training generalized multi-query transformer models from multi-head checkpoints. *arXiv preprint arXiv:2305.13245*, 2023.

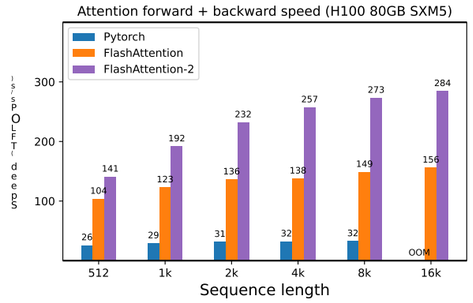
[2] Iz Beltagy, Matthew E Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.



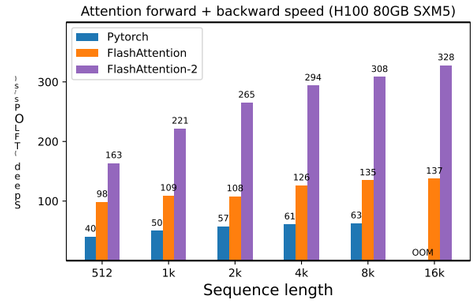
(a) Without causal mask, head dimension 64



(b) Without causal mask, head dimension 128



(c) With causal mask, head dimension 64



(d) With causal mask, head dimension 128

图7：H100 GPU上的注意力前向+后向速度

致谢

我们感谢Phil Tillet和Daniel Haziza，他们在Triton [17]和xformers库 [10]中实现了FlashAttention的版本。FlashAttention-2的灵感来源于不同注意力实现方式之间的思想交流。我们感谢Nvidia CUTLASS团队（特别是Vijay Thakkar、Cris Cecka、Haicheng Wu和Andrew Kerr）提供的CUTLASS库，尤其是CUTLASS 3.x版本，它为FlashAttention-2的实现提供了简洁的抽象和强大的构建模块。我们感谢Driss Guessous将FlashAttention集成到PyTorch中。FlashAttention-2受益于与Phil Wang、Markus Rabe、James Bradbury、Young-Jun Ko、Julien Launay、Daniel Hesslow、Michaël Benesty、Horace He、Ashish Vaswani和Erich Elsen的有益讨论。感谢斯坦福CRFM和斯坦福NLP提供的计算支持。我们感谢Dan Fu和Christopher Ré在设计硬件高效算法这一工作上的合作、建设性反馈和持续鼓励。我们感谢Albert Gu和Beidi Chen对本技术报告早期草稿提出的宝贵建议。

参考文献

[1] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. GQA：从多头检查点训练广义多查询Transformer模型。arXiv预印本 arXiv:2305.13245, 2023. [2] Iz Beltagy, Matthew E Peters, and Arman Cohan. Longformer：长文档Transformer。arXiv预印本 arXiv:2004.05150, 2020.

[3] Beidi Chen, Tri Dao, Eric Winsor, Zhao Song, Atri Rudra, and Christopher Ré. Scatterbrain: Unifying sparse and low-rank attention. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

[4] Krzysztof Marcin Choromanski, Valerii Likhosherstov, David Dohan, Xingyou Song, Andreea Gane, Tamas Sarlos, Peter Hawkins, Jared Quincy Davis, Afroz Mohiuddin, Lukasz Kaiser, et al. Rethinking attention with performers. In *International Conference on Learning Representations (ICLR)*, 2020.

[5] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*, 2022.

[6] Zhe Jia and Peter Van Sandt. Dissecting the Ampere GPU architecture via microbenchmarking. GPU Technology Conference, 2021.

[7] Zhe Jia, Marco Maggioni, Benjamin Staiger, and Daniele P Scarpazza. Dissecting the nvidia Volta GPU architecture via microbenchmarking. *arXiv preprint arXiv:1804.06826*, 2018.

[8] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are RNNs: Fast autoregressive transformers with linear attention. In *International Conference on Machine Learning*, pages 5156–5165. PMLR, 2020.

[9] Nikita Kitaev, Łukasz Kaiser, and Anselm Levskaya. Reformer: The efficient transformer. In *The International Conference on Machine Learning (ICML)*, 2020.

[10] Benjamin Lefaudeux, Francisco Massa, Diana Liskovich, Wenhan Xiong, Vittorio Caggiano, Sean Naren, Min Xu, Jieru Hu, Marta Tintore, Susan Zhang, Patrick Labatut, and Daniel Haziza. xformers: A modular and hackable transformer modelling library. `https://github.com/facebookresearch/xformers`, 2022.

[11] Maxim Milakov and Natalia Gimelshein. Online normalizer calculation for softmax. *arXiv preprint arXiv:1805.02867*, 2018.

[12] OpenAI. Gpt-4 technical report. *ArXiv*, abs/2303.08774, 2023.

[13] Markus N Rabe and Charles Staats. Self-attention does not need $O(n^2)$ memory. *arXiv preprint arXiv:2112.05682*, 2021.

[14] Aurko Roy, Mohammad Saffar, Ashish Vaswani, and David Grangier. Efficient content-based sparse attention with routing transformers. *Transactions of the Association for Computational Linguistics*, 9: 53–68, 2021.

[15] Noam Shazeer. Fast transformer decoding: One write-head is all you need. *arXiv preprint arXiv:1911.02150*, 2019.

[16] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-LM: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019.

[17] Philippe Tillet, Hsiang-Tsung Kung, and David Cox. Triton: an intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, pages 10–19, 2019.

[18] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

[19] Sinong Wang, Belinda Z Li, Madian Khabsa, Han Fang, and Hao Ma. Linformer: Self-attention with linear complexity. *arXiv preprint arXiv:2006.04768*, 2020.

[20] Manzil Zaheer, Guru Guruganesh, Kumar Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, et al. Big bird: Transformers for longer sequences. *Advances in Neural Information Processing Systems*, 33, 2020.

[3] 陈北地、道特里、埃里克·温瑟、宋钊、阿特里·鲁德拉与克里斯托弗·雷。《Scatterbrain：统一稀疏与低秩注意力》。收录于《神经信息处理系统进展》（NeurIPS），2021年。

[4] Krzysztof Marcin Choromanski, Valerii Likhosherstov, David Dohan, Xingyou Song, Andreea Gane, Tamas Sarlos, Peter Hawkins, Jared Quincy Davis, Afroz Mohiuddin, Lukasz Kaiser 等。重新思考注意力机制：Performers。收录于国际学习表征会议（ICLR），2020年。

[5] Tri Dao、Daniel Y. Fu、Stefano Ermon、Atri Rudra 和 Christopher Ré。FlashAttention：具有 IO 感知能力的快速且内存高效的精确实注意力机制。载于《神经信息处理系统进展》，2022年。

[6] Zhe Jia and Peter Van Sandt. 通过微基准测试剖析安培GPU架构。GPU技术大会，2021年。

[7] Zhe Jia, Marco Maggioni, Benjamin Staiger, and Daniele P Scarpazza. 通过微基准测试剖析NVIDIA Volta GPU架构。arXiv preprint arXiv:1804.06826, 2018.

[8] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are RNNs: Fast autoregressive transformers with linear attention. In International Conference on Machine Learning, pages 5156–5165. PMLR, 2020.

[9] Nikita Kitaev， ukasz Kaiser, and Anselm Levskaya. Reformer: 高效Transformer。发表于国际机器学习会议（ICML），2020年。

[10] Benjamin Lefaudeux, Francisco Massa, Diana Liskovich, Wenhan Xiong, Vittorio Caggiano, Sean Naren, Min Xu, Jieru Hu, Marta Tintore, Susan Zhang, Patrick Labatut, Daniel Haziza。xformers：一个模块化且可定制的Transformer建模库。https://github.com/facebookresearch/xformers, 2022。

[11] Maxim Milakov 与 Natalia Gimelshein。Softmax 的在线归一化器计算。arXiv 预印本 arXiv:1805.02867，2018年。

[12] OpenAI. GPT-4技术报告. ArXiv, abs/2303.08774, 2023.

[13] Markus N Rabe 与 Charles Staats。自注意力不需要 （ ^{2}) 内存。arXiv 预印本 arXiv:2112.05682，2021。

[14] Aurko Roy, Mohammad Saffar, Ashish Vaswani, David Grangier。基于内容的高效稀疏注意力与路由变换器。计算语言学协会汇刊，9：53–68，2021。

[15] Noam Shazeer。快速Transformer解码：一个写头足矣。arXiv预印本 arXiv:1911.02150，2019。

[16] Mohammad Shoeybi、Mostofa Patwary、Raul Puri、Patrick LeGresley、Jared Casper 和 Bryan Catanzaro。Megatron-LM：使用模型并行性训练数十亿参数语言模型。arXiv 预印本 arXiv:1909.08053，2019。

[17] Philippe Tillet、Hsiang-Tsung Kung 和 David Cox。Triton：一种用于分块神经网络计算的中间语言和编译器。载于《第三届 ACM SIGPLAN 机器学习与编程语言国际研讨会论文集》，第 10–19 页，2019 年。

[18] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez， ukasz Kaiser, and Illia Polosukhin. Attention is all you need. Advances in neural information processing systems, 30, 2017.

[19] Sinong Wang, Belinda Z Li, Madian Khabsa, Han Fang, and Hao Ma. Linformer: 具有线性复杂度的自注意力机制。arXiv preprint arXiv:2006.04768, 2020.

[20] Manzil Zaheer、Guru Guruganesh、Kumar Avinava Dubey、Joshua Ainslie、Chris Alberti、Santiago Ontanon、Philip Pham、Anirudh Ravula、Qifan Wang、Li Yang 等。Big bird：用于更长序列的 Transformer。神经信息处理系统进展，33，2020。